



RÉPUBLIQUE ALGÉRIENNE DÉMOCRATIQUE ET POPULAIRE



Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

UNIVERSITÉ DES SCIENCES ET DE LA TECHNOLOGIE HOUARI BOUMEDIENE

Faculté d'Electronique et d'Informatique

Département Informatique

MÉMOIRE DE MASTER

Spécialité

RÉSEAU ET SYSTÈMES DISTRIBUÉS

Thème

Conception et déploiement d'une infrastructure
Cloud privée hyperconvergée basée sur
des technologies open source

Proposé et Dirigé par :

Mr. Neffah Mohammed

Prof. MERAZKA Fatiha

Réalisé et présenté par :

Mr. Bouzara Zakaria

Mr. Ouacherine Ilyes

Devant le jury composé de :

M

M

Soutenu le : ../../2026

Binôme N° 18/2026

Remerciements

Votre texte ici...

Table des matières

Table des matières	3
Table des figures	4
Liste des tableaux	5
Liste des abréviations	6
Introduction générale	1
I Présentation du Contexte	2
I.1 Introduction	3
I.2 Présentation de Sonatrach	3
I.3 Infrastructure Existante et Contexte Technologique	3
I.3.1 Infrastructure actuelle de Sonatrach	3
I.3.2 Les besoins en infrastructure légère pour les zones de test	4
I.4 Problématique	5
I.5 Objectifs du Projet	5
I.5.1 Objectif général	5
I.5.2 Objectifs spécifiques	5
I.6 Périmètre et Limitations du Projet	6
I.7 Conclusion	7
II État de l'Art et Concepts Théoriques	8
II.1 Introduction	9
II.2 L'Hyperconvergence (HCI)	9
II.2.1 Définition	9
II.2.2 Avantages de l'hyperconvergence	9
II.2.3 HCI vs Infrastructure Traditionnelle	10
II.3 Comparaison : Solutions Propriétaires vs Open Source	10
II.3.1 Solutions propriétaires	10
II.3.2 Solutions open source	10
II.4 Analyse des Solutions Open Source HCI	11
II.4.1 OpenStack	11
II.4.2 Harvester HCI	11
II.4.3 Proxmox VE	11
II.4.4 Tableau comparatif	12
II.5 Présentation des Technologies Retenues	12
II.5.1 Proxmox VE : L'hyperviseur	12
II.5.2 Ceph : Le stockage distribué	15
II.5.3 Open vSwitch (OVS) : La virtualisation réseau	17

II.6	Déploiement sur Infrastructure Physique Distribuée	20
II.7	Conclusion	21
III	Présentation du Contexte	22
III.1	Introduction	23
III.2	Vue Globale de l'Architecture	23
III.3	Partie I : Calcul	24
III.3.1	Infrastructure Physique	24
III.3.2	Architecture du Cluster Proxmox	25
III.3.2.1	Organisation du Cluster	25
III.3.2.2	Mécanisme de Quorum et Corosync	25
III.3.2.3	Haute Disponibilité	25
III.3.2.4	Matrice des Scénarios de Défaillance	26
III.3.3	Mécanisme d'Équilibrage Dynamique : CLBS	26
III.3.3.1	Contexte et Besoin	26
III.3.3.2	Base Algorithmique : CSLB	26
III.3.3.3	Modifications Apportées au CSLB	27
III.3.3.4	Algorithme de Décision (Pseudocode)	28
III.3.4	Application de Démonstration : Sona-Web	30
III.3.4.1	Architecture Applicative	30
III.3.4.2	Scénario de Validation HA	31
III.3.5	Architecture du Monitoring	32
III.3.6	Automatisation	32
III.3.6.1	Ansible : Périmètre et Playbooks	32
III.3.6.2	Pipeline PXE/TFTP	32
III.4	Partie II : Stockage	33
III.4.1	Architecture du Cluster Ceph	33
III.4.1.1	Composants du Cluster	33
III.4.1.2	Dimensionnement du Pool	34
III.4.1.3	Configuration CRUSH et Domaines de Défaillance	34
III.5	Partie III : Réseau	35
III.5.1	Contexte et Contraintes Réseau	35
III.5.2	Topologie Réseau	35
III.5.2.1	Deux Niveaux de Segmentation	35
III.5.2.2	Bridges Open vSwitch par Nœud	37
III.5.3	Plan d'Adressage	37
III.5.4	VM OPNsense : Routeur et Pare-feu Centralisé	38
III.5.4.1	Interfaces et Rôles	38
III.5.4.2	Haute Disponibilité d'OPNsense	38
III.5.5	Architecture du Pare-feu	38
III.5.5.1	Pare-feu OPNsense : Filtrage Inter-réseaux	38
III.5.5.2	Pare-feu Proxmox : Filtrage au Niveau Hyperviseur	39
III.6	Conclusion	40
IV	Implémentation de la Solution Proposée	41
IV.1	Introduction	42
IV.2	Partie I — Couche Calcul	42
IV.2.1	Préparation de l'Infrastructure Physique	42
IV.2.1.1	Installation de Proxmox VE sur les nœuds	42
IV.2.1.2	Configuration de la connectivité Internet via le laptop gateway	43

IV.2.2	Formation du cluster Proxmox et haute disponibilité	44
IV.2.2.1	Création et jonction au cluster	44
IV.2.2.2	Configuration de la migration à chaud	45
IV.2.3	Déploiement de la stack de monitoring	46
IV.2.3.1	Déploiement d’InfluxDB	47
IV.2.3.2	Déploiement de Prometheus	47
IV.2.3.3	Déploiement de Grafana	48
IV.2.4	Implémentation du mécanisme CLBS	50
IV.2.4.1	Structure du service	50
IV.2.4.2	Logique de décision	50
IV.2.4.3	Déploiement comme service systemd	50
IV.2.4.4	Validation du mécanisme	50
IV.2.5	Déploiement de l’application Sona-Web	51
IV.2.5.1	Création des machines virtuelles	51
IV.2.5.2	Validation de l’application	51
IV.2.6	Automatisation avec Ansible	52
IV.2.6.1	Environnement et structure du projet	52
IV.2.6.2	Inventaire	52
IV.2.6.3	Playbook de provisionnement	53
IV.2.6.4	Provisionnement en lot et pipeline PXE/TFTP	53
IV.3	Partie II — Couche Stockage	53
IV.3.1	Déploiement du cluster Ceph	53
IV.3.1.1	Initialisation de Ceph	53
IV.3.1.2	Déploiement des MON et MGR	54
IV.3.1.3	Déploiement des OSDs	54
IV.3.1.4	Création du pool de stockage	54
IV.3.2	Validation de la tolérance aux pannes	56
IV.4	Partie III — Couche Réseau	56
IV.4.1	Configuration Open vSwitch et tunnels VxLAN	56
IV.4.1.1	Installation d’Open vSwitch	56
IV.4.1.2	Configuration complète des interfaces sur PVE-01	56
IV.4.2	Déploiement de la VM OPNsense	61
IV.4.2.1	Création et interfaces réseau	61
IV.4.2.2	NAT sortant	61
IV.4.3	Configuration du service DHCP	62
IV.4.4	Validation de la haute disponibilité réseau et applicative	62
IV.5	Conclusion	63

Table des figures

III.1 Architecture Globale de l'Infrastructure HCI	24
III.3 Logique de Décision du Mécanisme CLBS	29
III.4 Architecture Applicative Sona-Web	31
III.6 Topologie Réseau et Plan VxLAN	36
III.7 Architecture du Pare-feu : OPNsense et Proxmox Firewall	39
IV.1 Interface web Proxmox au premier login sur PVE-01	43
IV.10 Vue du cluster Proxmox montrant les trois nœuds en ligne	45
IV.11 Avant la migration live d'une machine virtuelle	46
IV.12 Après la migration live d'une machine virtuelle	46
IV.13 Explorateur InfluxDB montrant les métriques reçues depuis Proxmox	47
IV.15 Dashboard Grafana Infrastructure pour les nœuds Proxmox	49
IV.18 Dashboard ou logs montrant une migration déclenchée par CLBS	51
IV.33 État du cluster Ceph montrant les MON et OSD	55
IV.34 État du cluster Ceph montrant le pool RBD	55
IV.38 Configuration des interfaces de la VM OPNsense	61
IV.40 Chronologie des événements lors d'une panne d'un nœud physique	62

Liste des tableaux

III.1 Dimensionnement physique des serveurs Dell PowerEdge R720	25
III.2 Adresses IP de management des nœuds Proxmox	25
III.3 Matrice des scénarios de défaillance et comportements attendus	26
III.4 Répartition du stockage Ceph par usage	34
III.5 Paramètres CRUSH appliqués automatiquement par Proxmox	34
III.6 Bridges OVS et tunnels VxLAN par nœud Proxmox	37
III.7 Adressage IP statique des nœuds Proxmox par réseau logique	37
IV.1 Dashboards Grafana configurés	48
IV.2 Résumé des VxLAN utilisés dans l'infrastructure	60
IV.3 Interfaces de la VM OPNsense	61
IV.4 Plages DHCP configurées sur OPNsense	62

Liste des abréviations

API	: Application Programming Interface
BSD	: Berkeley Software Distribution
CLI	: Command Line Interface
CPU	: Central Processing Unit
CRUSH	: Controlled Replication Under Scalable Hashing
DRS	: Distributed Resource Scheduler
EC	: Erasure Coding
GUI	: Graphical User Interface
HA	: High Availability
HCI	: Hyper Converged Infrastructure
I/O	: Input / Output
IP	: Internet Protocol
ISO	: International Organization for Standardization (image disque)
KPI	: Key Performance Indicator
KVM	: Kernel-based Virtual Machine
LXC	: Linux Containers
MDS	: MetaData Server
MGR	: Manager
MON	: Monitor
NIC	: Network Interface Card
OSD	: Object Storage Daemon
OVS	: Open Virtual Switch
PC	: Personal Computer
PG	: Placement Group
PVE	: Proxmox Virtual Environment
QoS	: Quality of Service
RADOS	: Reliable Autonomic Distributed Object Store
RAM	: Random Access Memory
RBD	: RADOS Block Device
RSD	: Réseau Et Systèmes Distribués
SPOF	: Single Point Of Failure
SSH	: Secure Shell
TCO	: Total Cost of Ownership
TCP	: Transmission Control Protocol
UDP	: User Datagram Protocol
vCPU	: Virtual Central Processing Unit
VE	: Virtual Environment
VLAN	: Virtual Local Area Network
VM	: Virtual Machine
WAL	: Write-Ahead Log

Introduction générale

Chapitre I

Présentation du Contexte

I.1 Introduction

Ce chapitre vise à présenter le contexte général dans lequel s'inscrit notre projet de fin d'études. Nous y aborderons les fondements de l'hyper convergence (HCI), les enjeux liés à la virtualisation et à la gestion unifiée des ressources informatiques, ainsi que les besoins spécifiques de l'entreprise Sonatrach. Ce cadre introductif nous permettra de mieux comprendre les choix technologiques effectués et les objectifs fixés pour le déploiement d'une infrastructure Cloud privée hyperconvergente basée sur des solutions open source.

I.2 Présentation de Sonatrach

Sonatrach est une entreprise publique algérienne créée le 31 décembre 1963, peu après l'indépendance de l'Algérie. Elle a été fondée dans le but de gérer, développer et exploiter les ressources hydrocarbures du pays, qui représentent une richesse stratégique pour l'économie nationale. À ses débuts, Sonatrach se consacrait essentiellement au transport et à la commercialisation du pétrole brut extrait en Algérie. [1]

Avec le temps, Sonatrach a évolué pour devenir la plus grande entreprise en Afrique dans le domaine des hydrocarbures. Elle intervient aujourd'hui dans l'ensemble de la chaîne de valeur pétrolière et gazière : exploration, production, transport, transformation et commercialisation. L'entreprise est également active dans la pétrochimie et les énergies renouvelables.

Sonatrach entretient de nombreuses collaborations internationales avec des compagnies pétrolières et gazières étrangères, impliquées dans de grands projets de partenariat via des joint-ventures et accords technologiques avec des entreprises européennes, américaines, asiatiques et africaines. Ces coopérations ont permis un transfert de savoir-faire et de technologie vers l'Algérie.

Le projet est réalisé au sein du Centre d'innovation de Sonatrach. Cette direction joue un rôle stratégique dans l'innovation technologique et l'optimisation des processus internes. Elle dispose d'infrastructures informatiques avancées, notamment des solutions de virtualisation et de stockage haute capacité, destinées à soutenir les travaux de simulation, de test et de pré-production.

I.3 Infrastructure Existante et Contexte Technologique

I.3.1 Infrastructure actuelle de Sonatrach

Sonatrach dispose actuellement d'une infrastructure hyperconvergente basée sur la solution propriétaire Nutanix, déployée à l'échelle nationale sur plusieurs wilayas et utilisée pour les environnements de production critiques. Cette solution est reconnue pour sa stabilité, ses performances et la richesse de ses fonctionnalités, notamment en matière de gestion centralisée, de haute disponibilité et de scalabilité.

Cependant, cette infrastructure présente des contraintes structurelles qui deviennent de plus en plus préoccupantes dans une perspective d'évolution à long terme :

- **Coûts de licences croissants** : les frais de licence Nutanix suivent une trajectoire haussière, représentant une charge financière significative et récurrente pour l'entreprise, indépendamment du niveau d'utilisation réel de la plateforme.
- **Dépendance technologique forte** : le recours exclusif à un éditeur propriétaire unique expose Sonatrach à un risque de vendor lock-in, limitant sa capacité à négocier les conditions contractuelles et à adopter librement de nouvelles technologies.
- **Manque de souveraineté technique** : les évolutions de la plateforme, les mises à jour et les décisions d'architecture sont entièrement soumises à la roadmap de l'éditeur, sans possibilité d'adaptation aux besoins spécifiques de l'entreprise.
- **Inadaptation aux petits déploiements** : pour les environnements de test, de développement et de pré-production, le déploiement de Nutanix sur de petits clusters de 3 à 4 nœuds est disproportionné par rapport aux besoins réels, tant en complexité qu'en coût.

Ces constats positionnent ce projet non pas comme un simple exercice académique, mais comme une réponse concrète à une problématique stratégique réelle : explorer et valider la viabilité d'une infrastructure hyperconvergée open source capable de répondre aux exigences de Sonatrach, aujourd'hui en environnement de test et demain potentiellement en production.

I.3.2 Les besoins en infrastructure légère pour les zones de test

Au-delà des besoins immédiats en environnements de test et de pré-production, Sonatrach a besoin d'une infrastructure qui réponde à des exigences de niveau entreprise, tout en s'affranchissant des contraintes des solutions propriétaires. Ces besoins couvrent plusieurs dimensions :

- **Fiabilité et haute disponibilité** : l'infrastructure doit garantir la continuité de service en cas de défaillance d'un ou plusieurs nœuds, avec basculement automatique des charges de travail sans intervention humaine.
- **Performance et scalabilité** : la solution doit pouvoir monter en charge progressivement, en ajoutant des nœuds au cluster sans interruption de service, pour accompagner la croissance des besoins de l'entreprise.
- **Gestion unifiée des ressources** : calcul, stockage et réseau doivent être administrés depuis une interface centralisée unique, réduisant la complexité opérationnelle et les besoins en compétences spécialisées multiples.
- **Équilibrage dynamique de charge** : à l'image du mécanisme DRS de VMware, la plateforme doit être capable de redistribuer automatiquement les charges de travail entre les nœuds en fonction de l'utilisation des ressources, optimisant ainsi l'efficacité globale du cluster.
- **Souveraineté et indépendance technologique** : en s'appuyant exclusivement sur des technologies open source, l'entreprise reprend le contrôle total de son infrastructure, avec la liberté d'auditer, de modifier et d'adapter la solution à ses besoins propres.
- **Réduction du coût total de possession (TCO)** : l'absence de licences propriétaires représente une économie substantielle sur le long terme, réallouable vers des investissements matériels ou humains.

I.4 Problématique

La problématique centrale de ce projet est la suivante :

Comment concevoir et déployer une infrastructure Cloud privée hyperconvergée, entièrement basée sur des technologies open source, capable de répondre aux exigences de fiabilité, de performance et de scalabilité d’une grande entreprise industrielle, tout en constituant une alternative crédible et souveraine aux solutions propriétaires de type Nutanix ?

Cette problématique soulève plusieurs questions techniques structurantes :

- Comment concevoir un système de stockage distribué, redondant et performant garantissant l’intégrité des données en cas de défaillance d’un nœud ?
- Comment assurer le basculement et l’équilibrage automatique des charges de travail dans un cluster à plusieurs nœuds, sans perte de données ni interruption prolongée de service ?
- Comment segmenter et isoler strictement les flux réseau au sein d’une infrastructure hyperconvergée, en séparant les trafics de gestion, de réplication, de migration et de production ?
- Dans quelle mesure une solution open source peut-elle répondre aux exigences d’une infrastructure de niveau production dans le contexte d’une grande entreprise ?

I.5 Objectifs du Projet

I.5.1 Objectif général

Concevoir et déployer une infrastructure Cloud privée hyperconvergée, entièrement open source, sur trois serveurs physiques Dell PowerEdge R720 mis à disposition par Sonatrach, offrant des garanties de fiabilité, de performance et d’évolutivité comparables aux solutions propriétaires de type Nutanix, et constituant une alternative stratégique à moyen et long terme.

I.5.2 Objectifs spécifiques

- **Déployer un cluster de trois nœuds physiques à haute disponibilité** sur les serveurs Dell PowerEdge R720 fournis par Sonatrach, avec gestion centralisée, administration simplifiée et support de la migration à chaud.
- **Configurer Proxmox VE sur chaque serveur physique** en exploitant pleinement les ressources matérielles disponibles (256 GB RAM, 5 disques SAS par nœud).
- **Concevoir et mettre en place un système de stockage distribué Ceph** en utilisant les disques SAS 600 GB de chaque nœud comme OSD, et les disques 300 GB pour le système d’exploitation et les métadonnées Ceph (WAL/DB).

- **Implémenter une segmentation réseau avancée** via Open vSwitch, en configurant une architecture multi-VLAN séparant les flux de gestion, de réplication, de migration et de production sur l'infrastructure physique réelle.
- **Développer un mécanisme d'équilibrage dynamique de la charge (DRS)**, capable de redistribuer automatiquement les machines virtuelles selon les métriques de performance en temps réel.
- **Valider la solution** à travers un scénario de haute disponibilité démontrant la tolérance aux pannes sans perte de données ni interruption de service.
- **Fournir une base d'infrastructure évolutive**, accompagnée d'une documentation technique et d'une conception architecturale réutilisable à plus grande échelle au sein de Sonatrach.

I.6 Périmètre et Limitations du Projet

Ce projet vise le déploiement d'un cluster hyperconvergé open source destiné aux environnements de test, de développement et de pré-production. L'infrastructure est déployée sur un cluster physique de trois serveurs Dell PowerEdge R720 mis à disposition par Sonatrach, offrant les ressources matérielles nécessaires à l'exécution de l'hyperviseur Proxmox VE en *bare-metal*, du stockage distribué Ceph et des VMs de démonstration.

Le passage à une architecture physique distribuée lève la contrainte majeure du point de défaillance unique (SPOF) matériel. Néanmoins, le périmètre de ce projet se limite volontairement à un environnement de laboratoire structuré, ce qui implique les contraintes suivantes :

- **Échelle du cluster (Scale)** : Bien que le cluster repose sur trois serveurs physiques de classe entreprise offrant une excellente capacité de calcul (256 Go de RAM par nœud), il représente la configuration minimale recommandée (3 nœuds) pour maintenir un quorum stable avec Corosync et Ceph.
- **Capacité de stockage** : La taille du pool Ceph est techniquement contrainte par les disques physiques alloués aux OSDs (3 disques de 600 Go par serveur). Avec une réplication de facteur 3, le cluster offre environ 1,8 To d'espace utilisable. Cela rend cette infrastructure parfaitement adaptée aux charges de test et de développement, mais insuffisante pour des *workloads* de données à très grande échelle.
- **Absence de multitenancy** : Proxmox VE, dans la configuration mise en œuvre, est orienté Cloud privé. La gestion *multi-tenant* stricte (isolation forte entre différents clients ou départements totalement hermétiques), propre aux environnements Cloud publics, n'entre pas dans le périmètre de ce projet.
- **Maintenance à long terme** : La gestion opérationnelle continue de l'infrastructure (cycle de mises à jour, configuration d'un serveur Proxmox Backup Server externe sécurisé, gestion des pannes matérielles physiques) n'est pas couverte par la phase de déploiement de ce projet et constitue une perspective d'évolution naturelle pour l'équipe d'exploitation.

I.7 Conclusion

Dans ce chapitre, nous avons présenté le contexte général du projet, les besoins identifiés au sein de Sonatrach, et la problématique qui motive notre démarche. Nous avons défini les objectifs, le périmètre et les limites du projet. Ce cadre nous permettra d'aborder, dans le chapitre suivant, l'état de l'art des solutions disponibles et les fondements théoriques de l'hyperconvergence, de la virtualisation et des technologies retenues.

Chapitre II

État de l'Art et Concepts Théoriques

II.1 Introduction

Dans un contexte de transformation numérique accélérée, les organisations scientifiques et industrielles cherchent des solutions d'infrastructure à la fois performantes, flexibles et économiques. Ce chapitre présente les concepts théoriques fondamentaux sur lesquels repose notre projet : l'hyperconvergence, la virtualisation, le stockage distribué et la virtualisation du réseau. Il dresse également un état de l'art comparatif des solutions disponibles, afin de justifier les choix technologiques retenus.

II.2 L'Hyperconvergence (HCI)

II.2.1 Définition

L'hyperconvergence (HCI, Hyper-Converged Infrastructure) est une architecture informatique qui regroupe, dans un même système, les ressources de calcul, de stockage et de réseau. Ces ressources sont pilotées par une couche logicielle centralisée, supprimant ainsi la nécessité de gérer des équipements spécialisés distincts pour chaque fonction. [2]

L'infrastructure hyperconvergée repose sur trois composantes fondamentales :

- **Le calcul** : des processeurs virtuels (vCPU) alloués dynamiquement aux machines virtuelles.
- **Le stockage** : un système de stockage distribué logiciel, agrégé à partir des disques locaux de chaque nœud.
- **Le réseau** : un switch virtuel logiciel permettant la segmentation et l'isolation du trafic réseau.

II.2.2 Avantages de l'hyperconvergence

L'approche HCI présente plusieurs avantages par rapport aux architectures traditionnelles à trois niveaux (compute, storage, network séparés) :

- **Simplicité d'administration** : une interface unique pour gérer l'ensemble des ressources.
- **Réduction des coûts matériels** : pas besoin de baies de stockage ou de switches réseau dédiés.
- **Évolutivité horizontale** : l'ajout d'un nœud augmente simultanément la capacité de calcul, de stockage et de réseau.
- **Haute disponibilité native** : la réplication des données entre les nœuds assure la continuité de service en cas de panne.

II.2.3 HCI vs Infrastructure Traditionnelle

Critère	Infrastructure Traditionnelle	Infrastructure HCI
Gestion	Silos séparés (compute, storage, network)	Interface unifiée
Coût initial	Elevé (équipements spécialisés)	Réduit (commodity hardware)
Évolutivité	Verticale (upgrade de matériel)	Horizontale (ajout de nœuds)
Complexité	Haute	Faible
Haute disponibilité	Optionnelle et complexe	Native

Tableau 2-1 : Comparaison du HCI avec Infrastructure Traditionnelle

II.3 Comparaison : Solutions Propriétaires vs Open Source

II.3.1 Solutions propriétaires

Les solutions propriétaires telles que **VMware vSAN** ou **Nutanix** offrent un support professionnel, des fonctionnalités avancées et une interface utilisateur soignée. Cependant, elles présentent plusieurs limites importantes dans notre contexte :

- **Coûts élevés** : les licences et la maintenance annuelle représentent un investissement significatif, difficilement justifiable pour des environnements de test.
- **Dépendance fournisseur (Vendor Lock-in)** : la migration vers une autre solution est souvent complexe et coûteuse.
- **Faible flexibilité** : les environnements propriétaires offrent peu de liberté pour personnaliser ou adapter l'infrastructure.

II.3.2 Solutions open source

Les solutions open source présentent plusieurs avantages stratégiques :

- **Indépendance technologique** : aucune dépendance vis-à-vis d'un éditeur.
- **Absence de coûts de licence** : seuls les coûts d'infrastructure matérielle et de main-d'œuvre sont à prévoir.
- **Transparence et auditabilité** : le code source est accessible et auditable.
- **Communauté active** : documentation abondante et mises à jour régulières.

Dans un contexte de zones de test et de petits déploiements, l'approche open source est clairement la plus adaptée.

II.4 Analyse des Solutions Open Source HCI

Le critère principal de sélection est la légèreté et la capacité à fonctionner de manière stable sur des clusters de 3 à 4 nœuds.

II.4.1 OpenStack

OpenStack est une plateforme cloud open source très complète et standardisée, utilisée par de nombreuses grandes entreprises et fournisseurs cloud. Elle propose une large gamme de services : calcul (Nova), stockage objet (Swift), stockage en bloc (Cinder), réseau (Neutron), identité (Keystone), etc.

Cependant, OpenStack présente des inconvénients majeurs pour notre cas d'usage :

- **Complexité de déploiement** : l'installation et la configuration d'OpenStack nécessitent une expertise poussée et prennent un temps considérable.
- **Consommation de ressources élevée** : les services de base d'OpenStack consomment à eux seuls plusieurs Go de RAM, avant même de démarrer des VMs de production.
- **Inadapté aux petits clusters** : OpenStack est conçu pour des déploiements de grande envergure. Sur 3 nœuds, ses performances et sa stabilité sont compromises.

II.4.2 Harvester HCI

Harvester est une solution HCI open source développée par SUSE, basée sur Kubernetes et KubeVirt. Elle propose une interface moderne et une intégration native avec Rancher pour la gestion de clusters Kubernetes. [5]

Ses limites dans notre contexte :

- **Consommation en RAM et CPU élevée** : Harvester nécessite des ressources importantes pour faire fonctionner la couche Kubernetes sous-jacente, avant même d'allouer des ressources aux VMs.
- **Maturité relative pour les VMs classiques** : Harvester est davantage orienté vers les workloads conteneurisés. La gestion de VMs Linux/Windows classiques est moins optimisée que sur une solution dédiée.
- **Courbe d'apprentissage** : la maîtrise de Kubernetes est un prérequis pour administrer efficacement Harvester. [4]

II.4.3 Proxmox VE

Proxmox Virtual Environment (VE) est une plateforme de virtualisation open source basée sur KVM (pour les VMs) et LXC (pour les conteneurs). Elle propose une interface web centralisée, un support natif de Ceph pour le stockage distribué, et une gestion intégrée du clustering et de la haute disponibilité. [6]

Ses atouts pour notre cas d'usage sont nombreux :

- **Légèreté** : moins de 1 Go de RAM consommé à vide sur le nœud, laissant l'essentiel des ressources aux VMs.
- **Déploiement rapide** : un cluster de 3 nœuds est opérationnel en quelques heures.
- **Interface web intuitive** : la console Proxmox (accessible sur le port 8006) centralise la gestion des VMs, conteneurs, stockage, réseau et haute disponibilité.
- **Intégration native de Ceph** : Proxmox intègre nativement les outils de gestion Ceph, simplifiant considérablement le déploiement du stockage distribué.
- **Communauté active** : documentation officielle complète et forums très actifs.

II.4.4 Tableau comparatif

Critère	OpenStack	Harvester	Proxmox VE
Légèreté	Faible	Moyenne	Elevée
Facilité de déploiement	Difficile	Moyenne	Facile
Adapté aux petits clusters (3 nœuds)	Non	Partiel	Oui
Intégration stockage distribué	Via Cinder/Ceph	Via Longhorn	Ceph natif
Gestion des VMs classiques	Bonne	Moyenne	Excellente
Maturité	Très mature	En cours	Mature
Coût de licence	Gratuit	Gratuit	Gratuit (Enterprise payant)

Tableau 2-2 : OpenStack vs Harvester vs Proxmox VE

Proxmox VE est la solution la mieux adaptée à nos contraintes. Elle sera retenue comme hyperviseur principal du projet.

II.5 Présentation des Technologies Retenues

II.5.1 Proxmox VE : L'hyperviseur

Présentation générale

Proxmox Virtual Environment (VE) est une plateforme de virtualisation open source basée sur un noyau Linux Debian modifié. Elle supporte deux types de virtualisation : **KVM** (*Kernel-based Virtual Machine*) pour les machines virtuelles complètes, compatibles avec tout système d'exploitation (Linux, Windows, BSD), et **LXC** (*Linux Containers*) pour les conteneurs légers partageant le noyau de l'hôte. L'ensemble est administré via une interface web centralisée accessible sur le port 8006, qui unifie la gestion des VMs, conteneurs, stockage, réseau et haute disponibilité depuis un point d'entrée unique.

La pile de virtualisation : KVM, QEMU et libvirt

La virtualisation des machines dans Proxmox repose sur une pile logicielle à trois niveaux qui collaborent étroitement.

KVM — Kernel-based Virtual Machine

KVM est un module du noyau Linux (chargé via `kvm.ko` et `kvm_intel.ko` ou `kvm_amd.ko` selon le processeur) qui transforme le noyau Linux lui-même en hyperviseur de type 1. KVM expose au système des périphériques virtuels (`/dev/kvm`) et exploite les extensions matérielles de virtualisation du processeur — **Intel VT-x** ou **AMD-V** — pour exécuter le code des VMs directement sur le CPU physique sans émulation. Cela signifie que les instructions non privilégiées de la VM s'exécutent à la vitesse native du processeur, tandis que les instructions privilégiées (accès mémoire, I/O) sont interceptées et gérées par le noyau hôte. Le résultat est une performance CPU très proche du bare metal, typiquement entre 95% et 99% des performances natives mesurées en benchmarks.

QEMU — Quick EMUlator

QEMU est l'émulateur de machine qui accompagne KVM. Il opère en espace utilisateur et assure plusieurs fonctions essentielles :

- Il émule les **périphériques** de la VM : carte réseau virtuelle (VirtIO, e1000, RTL8139), contrôleur de disque (VirtIO SCSI, IDE, SATA), carte graphique, USB, BIOS ou UEFI (via OVMF). Chaque VM en cours d'exécution correspond à un processus `qemu-system-x86_64` sur l'hôte.
- Il gère le **CPU virtuel (vCPU)** : le nombre de cœurs, le modèle de CPU exposé à la VM (host-passthrough, kvm64, Haswell, Broadwell, etc.), et les flags CPU visibles par le système invité.
- Il assure la **gestion mémoire** : allocation de RAM, support du *Memory Ballooning* (ajustement dynamique de la RAM d'une VM en cours d'exécution), et des *Huge Pages* pour réduire la pression sur le TLB.
- Il fournit les **interfaces de migration** utilisées pour la live migration : QEMU implémente le protocole de transfert de l'état complet d'une VM (mémoire, état CPU, état des périphériques) vers un autre hôte sans interruption de service, fonctionnalité centrale pour notre mécanisme DRS.

Lorsque KVM est disponible, QEMU délègue l'exécution du code de la VM à KVM et n'émule que les périphériques. C'est cette combinaison KVM+QEMU qui constitue l'hyperviseur réel sous Proxmox.

Gestion du cluster : Corosync et le quorum

La gestion du cluster Proxmox repose sur **Corosync**, un framework de communication de cluster qui assure le consensus entre les nœuds via un mécanisme de *quorum*. Corosync envoie des messages *heartbeat* à intervalles réguliers entre les nœuds (sur notre VLAN 20 dédié) pour détecter les défaillances. Un cluster Proxmox nécessite un minimum de **3 nœuds** pour garantir un quorum stable.

Le quorum définit le nombre minimum de nœuds qui doivent être en accord pour que le cluster soit considéré comme opérationnel. Avec 3 nœuds, la majorité est de 2 : si un nœud tombe, les 2 restants ont toujours quorum et le cluster continue de fonctionner. Si

2 nœuds tombent simultanément, le nœud isolé n'a plus quorum et se met en retrait pour éviter le *split-brain*, une situation où deux parties du cluster croient chacune être la seule à fonctionner et modifient les données de manière concurrente.

Haute disponibilité (HA Manager)

Proxmox intègre un gestionnaire de haute disponibilité natif composé de deux services :

- **pve-ha-crm** (*Cluster Resource Manager*) : le cerveau centralisé du HA. Il surveille l'état de toutes les ressources protégées (VMs, conteneurs) à l'échelle du cluster et prend les décisions de basculement.
- **pve-ha-lrm** (*Local Resource Manager*) : l'agent local sur chaque nœud. Il exécute les ordres du CRM (démarrer, arrêter, migrer une VM) et remonte l'état des ressources locales.

Lorsqu'un nœud devient inaccessible (perte de heartbeat Corosync), le CRM déclenche le **fencing** (isolation forcée du nœud défaillant via le watchdog matériel ou logiciel), puis ordonne le redémarrage des VMs protégées sur les nœuds survivants. Le délai typique de basculement est de 60 à 120 secondes. Dans notre infrastructure, toutes les VMs applicatives (HAProxy, Apache, PostgreSQL) sont enregistrées dans le HA Manager avec leurs disques stockés sur Ceph RBD, ce qui leur permet d'être redémarrées sur n'importe quel nœud du cluster sans déplacement de données.

Live Migration

La migration à chaud (*live migration*) permet de déplacer une VM en cours d'exécution d'un nœud physique vers un autre sans interruption de service perceptible par l'utilisateur. Proxmox implémente ce mécanisme via QEMU en trois phases :

1. **Pré-copie mémoire** : QEMU copie itérativement les pages mémoire de la VM vers le nœud cible pendant que la VM continue de fonctionner. Les pages modifiées pendant la copie (*dirty pages*) sont recopiées lors de passes successives.
2. **Basculement** : lorsque le volume de dirty pages devient suffisamment faible (quelques dizaines de millisecondes de transfert), la VM est brièvement suspendue, les dernières pages sont transférées, et la VM reprend sur le nœud cible.
3. **Nettoyage** : les ressources de la VM sont libérées sur le nœud source.

Cette fonctionnalité est le mécanisme central exploité par notre service **DRS** (*Dynamic Resource Scheduler*), qui déclenche automatiquement des live migrations pour équilibrer la charge CPU entre les nœuds. Le trafic de migration transite exclusivement sur notre **VLAN 50** dédié (10.10.50.0/24), isolé des autres flux pour garantir la bande passante nécessaire sans impacter les workloads de production.

Interface de gestion et API REST

Proxmox expose une **API REST complète** sur le port 8006 (HTTPS), utilisée aussi bien par l'interface web que par des outils externes. Chaque opération réalisable depuis l'interface graphique est accessible via l'API : création de VMs, gestion du stockage, déclenchement de migrations, consultation des métriques en temps réel. Notre service DRS exploite directement cette API pour interroger l'état des nœuds, lister les VMs en cours d'exécution, et

déclencher les migrations programmatiquement via des appels HTTP authentifiés par ticket de session.

Récapitulatif de la pile technique Proxmox

Composant	Rôle
KVM (<code>kvm.ko</code>)	Hyperviseur noyau, exécution native du code VM via VT-x / AMD-V
QEMU (<code>qemu-system-x86_64</code>)	Émulation des périphériques, gestion mémoire, live migration
LXC	Conteneurs Linux légers partageant le noyau hôte
Corosync	Heartbeat cluster, consensus, détection de panne
pve-ha-crm / lrm	Gestionnaire de haute disponibilité, fencing, basculement automatique
API REST (port 8006)	Interface de pilotage programmatique, exploitée par le DRS

II.5.2 Ceph : Le stockage distribué

Ceph est une solution libre de stockage distribuée (Software-Defined Storage) qui s'impose aujourd'hui comme une référence dans le domaine des infrastructures open source. Conçu pour répondre aux besoins de fiabilité, d'extensibilité et de performance, Ceph constitue le choix idéal pour assurer la couche de stockage d'une infrastructure hyperconvergée telle que celle que nous proposons. Sa capacité à fonctionner sur du matériel standard (serveurs x86) et son architecture auto-réparatrice en font un composant central de notre projet.

Architecture de Ceph

L'architecture de Ceph repose sur un composant fondamental appelé RADOS (Reliable Autonomic Distributed Object Store). RADOS constitue le cœur du système, chargé de la gestion, de la réplication et de la répartition des données sur l'ensemble du cluster. Autour de ce cœur gravitent plusieurs types de démons qui interagissent pour former un cluster cohérent et performant :

- **Ceph Monitor (MON)** : Les Moniteurs sont les arbitres d'état du cluster. Ils maintiennent une carte maîtresse (*cluster map*) décrivant la topologie complète du cluster et assurent le consensus entre les nœuds via un mécanisme de quorum. En cas de désaccord, la majorité des Monitors décide de l'état authoritative du cluster, évitant ainsi les situations de *split-brain*.
- **Ceph Manager (MGR)** : Le Manager agit comme un chef d'orchestre. Il collecte toutes les métriques et informations d'état du cluster en temps réel (utilisation du stockage, performances, statistiques). Il expose ces données via une interface web de monitoring et les met à disposition des systèmes de gestion externes, facilitant ainsi l'administration et la supervision.
- **Ceph OSD (Object Storage Daemon)** : L'OSD est le démon qui gère les périphériques de stockage physiques ou logiques (disques, partitions, volumes logiques). Chaque OSD est responsable de la lecture et de l'écriture des données sur son disque.

Les OSDs communiquent entre eux pour assurer la réplication, le rééquilibrage et la réparation des données en cas de panne. Dans une architecture hyperconvergée, les OSDs résident sur les mêmes nœuds que l'hyperviseur (Proxmox), chaque nœud contribuant ainsi au stockage global.

- **Ceph Metadata Server (MDS)** : Ce démon est spécifique au système de fichiers distribué CephFS. Il gère les métadonnées des fichiers (noms, permissions, arborescences). Pour notre projet, qui utilise principalement le stockage en mode bloc pour les machines virtuelles, ce composant n'est pas requis.

La gestion intelligente des données : CRUSH et Placement Groups

L'intelligence de Ceph réside dans sa manière unique de gérer l'emplacement des données. Contrairement aux systèmes de stockage traditionnels qui utilisent une table de correspondance centralisée, Ceph utilise un algorithme appelé CRUSH (Controlled Replication Under Scalable Hashing). Cet algorithme permet à un client de calculer mathématiquement l'emplacement exact d'un objet (ou de sa réplique) en se basant sur la carte CRUSH, sans avoir à interroger un serveur central. Cette approche élimine un point de défaillance unique et améliore considérablement la scalabilité.

La carte CRUSH est une structure hiérarchique qui définit l'organisation physique du cluster. L'administrateur peut y définir des "domaines de défaillance" (nœuds, racks, salles, data centers). Ceph utilise ensuite ces informations pour placer les répliques des données sur des nœuds distincts, garantissant ainsi que la perte d'un équipement n'entraîne pas de perte de données.

Pour optimiser la gestion des milliers d'objets, Ceph les regroupe dans des unités logiques appelées Placement Groups (PGs). Les PGs constituent une abstraction intermédiaire entre l'objet et l'OSD, permettant un équilibrage efficace des données et une meilleure performance du cluster.

Les modes de protection des données

Pour assurer la fiabilité et la disponibilité des données, Ceph propose deux mécanismes principaux, adaptés à différents besoins :

- **Réplication** : C'est le mode le plus simple et le plus performant. Chaque objet est dupliqué en N copies (généralement 3) et chaque copie est placée sur des OSDs différents, dans des domaines de défaillance distincts. En cas de panne d'un OSD ou d'un nœud, les données restent disponibles sur les autres répliques. Ce mode offre une reconstruction rapide et est idéal pour les charges de travail exigeant de hautes performances en écriture, comme les disques de machines virtuelles. Dans notre projet, nous utiliserons une réplication de facteur 3 pour garantir une haute disponibilité sur le cluster.
- **Erasure Coding (Codage d'effacement)** : Ce mode est plus efficace en termes d'espace de stockage. Les données sont découpées en K fragments de données, puis encodées avec M fragments de parité, pour un total de K+M fragments répartis sur différents OSDs. Ce mode permet de tolérer la perte de M fragments tout en reconstruisant les données initiales. Il est souvent utilisé pour les données froides ou l'archivage, car il nécessite moins d'espace que la réplication (ex : 1,5x l'espace brut contre 3x pour la réplication), mais il est plus coûteux en calcul lors des opérations de reconstruction.

Intégration de Ceph dans une architecture hyperconvergée

L'intégration de Ceph avec Proxmox VE, notre hyperviseur choisi, est native et profondément intégrée. Proxmox offre une interface unifiée qui permet de créer et de gérer un cluster Ceph directement depuis son interface web, simplifiant considérablement les opérations de déploiement et de maintenance.

Dans notre architecture, chaque nœud Proxmox joue un double rôle :

- **Rôle de calcul** : Il exécute les machines virtuelles via KVM.
- **Rôle de stockage** : Il contribue au stockage global via ses OSDs Ceph.

Cette convergence des rôles est l'essence même de l'hyperconvergence. Les volumes de stockage des VMs sont fournis via Ceph RBD (RADOS Block Device), qui expose des périphériques de stockage en bloc sur le réseau. Ces volumes sont distribués et répliqués sur l'ensemble du cluster, assurant ainsi la persistance des données même en cas de panne d'un nœud.

Les avantages apportés par Ceph dans notre infrastructure sont multiples :

1. **Évolutivité horizontale** : L'ajout d'un nouveau nœud apporte simultanément de la puissance de calcul et de la capacité de stockage. Ceph rééquilibre automatiquement les données, permettant une croissance linéaire et sans interruption.
2. **Haute disponibilité** : L'absence de point de défaillance unique (SPOF) et la répliquation des données sur plusieurs nœuds garantissent que le système de stockage reste opérationnel même en cas de panne d'un ou plusieurs serveurs.
3. **Auto-réparation** : En cas de panne d'un disque ou d'un nœud, le cluster détecte l'incident et commence immédiatement à recréer les répliques manquantes sur les autres nœuds, assurant ainsi la pérennité des données sans intervention manuelle.
4. **Coût maîtrisé** : Ceph est open source et fonctionne sur du matériel standard (x86), ce qui le rend bien moins coûteux que les solutions propriétaires comme les SAN ou les appliances HCI propriétaires.
5. **Simplicité de gestion** : L'unification du stockage via Proxmox et les outils modernes comme `cephadm` (permettant un déploiement conteneurisé et simplifié) réduisent considérablement la complexité administrative.

II.5.3 Open vSwitch (OVS) : La virtualisation réseau

Présentation générale

Open vSwitch (OVS) est un commutateur logiciel multicouche open source, distribué sous licence Apache 2.0, conçu pour fournir une plateforme de commutation de qualité production dans les environnements de virtualisation. Il fonctionne selon deux composants principaux : `ovs-vswitchd`, le démon qui implémente le switch avec son module noyau Linux pour un transfert haute performance, et `ovsdb-server`, une base de données légère de configuration persistante. Parmi ses fonctionnalités natives figurent le modèle VLAN 802.1Q, le bonding avec ou sans LACP, la supervision via NetFlow et sFlow, la configuration QoS, ainsi que le

support natif des protocoles de tunneling GRE, VXLAN et Geneve.

Dans notre infrastructure, OVS joue le rôle de commutateur central de chaque nœud du cluster, assurant la segmentation de tout le trafic en flux distincts et isolés depuis un bridge virtuel unique par serveur.

Pourquoi segmenter le réseau dans un cluster HCI ?

La nécessité d'une segmentation réseau stricte découle de la coexistence, sur la même infrastructure physique, de trafics aux profils radicalement différents et incompatibles :

- La **réplication Ceph** génère un trafic continu à très haute bande passante entre les OSDs, pouvant saturer un réseau partagé.
- Les **heartbeats Corosync** sont extrêmement sensibles à la latence : tout délai ou perte de paquet peut déclencher un basculement non souhaité du cluster.
- Les **migrations à chaud** (live migration) produisent des pics de transfert ponctuels mais intenses, correspondant au déplacement complet de la mémoire vive d'une VM entre deux nœuds.
- Les **workloads applicatifs** génèrent un trafic variable et imprévisible, qui ne doit pas interférer avec les flux d'infrastructure.

Sans isolation stricte, ces flux en compétition se dégradent mutuellement et menacent la stabilité globale du cluster. Cette problématique est identique à celle que résout Nutanix avec la séparation entre le trafic CVM (Controller VM) et le trafic des VMs utilisateur. Notre architecture traduit cette logique en segments réseau dédiés gérés par OVS.

La conception de notre plan d'adressage a également été guidée par les recommandations officielles de la documentation Proxmox et les guidelines Ceph, qui préconisent explicitement la séparation du réseau cluster Ceph (réplication OSD-OSD) du réseau public Ceph (accès VM au stockage), ainsi que l'isolation du trafic Corosync sur une interface dédiée.

Architecture réseau à neuf segments logiques

Ces contraintes ont abouti à la définition de neuf segments réseau distincts, chacun correspondant à un type de trafic précis avec une plage d'adressage dédiée :

- **VXLAN 10** (Management — 192.168.10.0/24) : accès à l'interface web Proxmox, SSH et API. Faible volume mais disponibilité permanente requise.
- **VXLAN 20** (Corosync — 192.168.20.0/24) : heartbeats et quorum du cluster. Extrêmement sensible à la latence : toute perturbation peut déclencher un basculement non souhaité.
- **VXLAN 30** (Ceph Cluster — 10.10.30.0/24) : communications internes entre OSDs, réplication, rebalancing et recovery. Trafic continu à très haute bande passante, isolé conformément aux recommandations officielles Ceph.
- **VXLAN 40** (Ceph Public — 10.10.40.0/24) : accès des VMs au stockage Ceph via RBD. Séparé du VXLAN 30 pour éviter que les opérations internes Ceph ne dégradent les performances applicatives.

- **VXLAN 50** (Live Migration — 10.10.50.0/24) : migrations à chaud entre nœuds. Trafic soutenu et ponctuel, particulièrement sollicité par le mécanisme DRS que nous développons.
- **VXLAN 60** (VM Production — 172.16.60.0/24) : workloads applicatifs de production, dont Sona-Web. Point d'entrée utilisateur via HAProxy.
- **VXLAN 70** (VM Test/Dev — 172.16.70.0/24) : environnements de développement et validation, strictement isolés de la production.
- **VXLAN 80** (DMZ / External — 172.16.80.0/24) : services exposés vers l'extérieur de l'entreprise, protégés par pare-feu. HAProxy est le seul point d'entrée vers cette zone.
- **VXLAN 90** (Backup — 10.10.90.0/24) : transferts de sauvegarde et snapshots Proxmox, isolés pour ne pas impacter les workloads en production.

Contrainte d'infrastructure rencontrée et adaptation : adoption de VXLAN

L'architecture initiale reposait sur une segmentation par **VLANs 802.1Q**, nécessitant que le switch physique soit configuré en mode *trunk* pour propager les tags VLAN entre les trois nœuds. Cette approche est standard et parfaitement supportée par OVS.

Cependant, le déploiement sur l'infrastructure de Sonatrach a révélé une contrainte opérationnelle déterminante : les trois serveurs Dell R720 sont connectés à un switch administré dont la configuration est gérée exclusivement par l'équipe réseau de l'entreprise. Ce switch dispose déjà de VLANs définis (VLAN 10 pour le management, VLAN 20 pour un autre usage interne), et notre équipe ne dispose d'aucun droit d'administration pour modifier sa configuration, créer de nouveaux VLANs, ou reconfigurer les ports en mode trunk. Nos serveurs sont simplement connectés en mode access sur le VLAN 10.

Face à cette contrainte, nous avons adopté une approche alternative : le remplacement des VLANs 802.1Q par des tunnels **VXLAN** (*Virtual eXtensible LAN*, RFC 7348).

VXLAN : principe de fonctionnement

VXLAN est un protocole d'encapsulation réseau qui encapsule des trames Ethernet de niveau 2 dans des paquets UDP/IP de niveau 4, permettant de créer des réseaux virtuels superposés (*overlay networks*) par-dessus un réseau physique existant, sans nécessiter aucune modification de ce dernier.

Le principe est le suivant : lorsqu'une VM sur le nœud A souhaite communiquer avec une VM sur le nœud B à travers un réseau logique donné, OVS encapsule la trame Ethernet originale dans un paquet UDP à destination de l'adresse IP physique du nœud B. Ce paquet transite sur le réseau physique sous-jacent (notre VLAN 10 Sonatrach) comme un paquet IP ordinaire. À la réception, OVS sur le nœud B désencapsule le paquet et restitue la trame originale à la VM de destination. Le switch physique ne voit que du trafic UDP ordinaire et n'a aucune connaissance des réseaux virtuels qui transitent au-dessus de lui.

Chaque réseau logique est identifié par un **VNI** (*VXLAN Network Identifier*), un entier sur 24 bits, l'équivalent fonctionnel du tag VLAN. Notre architecture à neuf segments se traduit donc en neuf tunnels VXLAN avec neuf VNIs distincts (VNI 10 à VNI 90), créés et gérés nativement par OVS sur chacun des trois nœuds.

Impact sur l'architecture et bilan

Ce changement, contraint par l'environnement opérationnel de Sonatrach, ne remet pas en cause l'architecture réseau à neuf segments définie initialement. Les neuf segments logiques (management, Corosync, Ceph cluster, Ceph public, live migration, VM production, test/dev, DMZ, backup) sont intégralement conservés, chacun matérialisé par un VNI VXLAN dédié au lieu d'un tag VLAN. OVS assure la création, la gestion et l'isolation de ces tunnels de manière transparente et sans aucune intervention sur l'infrastructure réseau physique de l'entreprise.

Ce changement constitue en réalité une évolution positive sur plusieurs points. Premièrement, il renforce l'indépendance de l'infrastructure HCI vis-à-vis du réseau physique sous-jacent : le cluster peut fonctionner sur n'importe quel réseau IP sans prérequis de configuration. Deuxièmement, VXLAN est le protocole d'isolation réseau standard des infrastructures cloud modernes (AWS, Azure, GCP utilisent tous des overlays de type VXLAN ou similaire), ce qui renforce la représentativité technologique du projet par rapport aux déploiements de production actuels. Troisièmement, la capacité de 16 millions de VNIs, contre 4 094 VLANs 802.1Q, offre une scalabilité largement suffisante pour toute extension future du cluster.

La seule contrainte introduite est un surcoût d'encapsulation de 50 octets par paquet UDP, négligeable au regard des débits disponibles sur des liaisons Gigabit ou 10 Gigabit Ethernet. Les performances de réplication Ceph, des migrations à chaud et des heartbeats Corosync ne sont pas affectées de manière mesurable par cette encapsulation supplémentaire.

II.6 Déploiement sur Infrastructure Physique Distribuée

Contrairement à la conception initiale qui envisageait d'héberger l'ensemble du cluster sur un seul serveur via de la virtualisation imbriquée (nested virtualization), le projet a évolué vers un déploiement physique distribué, plus proche des conditions de production réelles. Sonatrach a mis à disposition trois serveurs Dell PowerEdge R720, sur lesquels Proxmox VE a été installé directement en mode bare metal.

Modèle de déploiement retenu

Le modèle de déploiement est organisé en deux niveaux :

- **Niveau 0 (Physique) :** chacun des trois serveurs Dell PowerEdge R720 héberge Proxmox VE directement en bare metal, sans couche de virtualisation intermédiaire. Chaque serveur constitue un nœud indépendant du cluster (PVE-01, PVE-02, PVE-03), avec ses propres ressources physiques (CPU, RAM, disques SAS).
- **Niveau 1 (VMs applicatives) :** le cluster Proxmox héberge directement les VMs de démonstration constituant l'application Sona-Web (HAProxy, Apache, PostgreSQL), les services de monitoring (InfluxDB, Grafana) et le service DRS.

Avantages du déploiement physique par rapport à la virtualisation imbriquée

Ce changement de paradigme apporte des améliorations significatives sur plusieurs dimensions :

- **Résilience réelle aux pannes matérielles** : dans l'architecture initiale à un seul serveur, une panne matérielle du serveur hôte aurait entraîné l'arrêt complet du cluster. Avec trois serveurs physiques distincts, la panne d'un nœud est absorbée par le mécanisme HA de Proxmox et le quorum Corosync, sans impact sur la disponibilité globale.
- **Performances réseau authentiques** : les communications inter-nœuds (réplication Ceph, heartbeats Corosync, migrations à chaud) transitent sur le réseau physique réel, ce qui permet de valider les performances dans des conditions proches de la production. Dans l'architecture initiale, ces flux transitaient par le bus mémoire interne d'un seul serveur, faussant les mesures de performance.
- **Représentativité des résultats** : les tests de haute disponibilité, d'équilibrage de charge et de performance du stockage Ceph sont réalisés sur une infrastructure physique réelle, ce qui renforce la valeur probatoire des résultats obtenus pour une éventuelle extension à la production.
- **Suppression de la complexité de la virtualisation imbriquée** : Le déploiement bare metal élimine entièrement la couche de virtualisation imbriquée (*nested virtualization*), supprimant les pertes de performance CPU liées à la double émulation, les contraintes de gestion des flags `nested-virt`, et les limitations imposées par l'hyperviseur hôte sur les fonctionnalités KVM avancées.

Caractéristiques du déploiement physique

- **Nombre de nœuds physiques** : 3 serveurs Dell PowerEdge R720
- **Modèle de déploiement** : Bare metal (Proxmox VE installé directement)
- **Adresse IP PVE-01** : 192.168.10.100
- **Adresse IP PVE-02** : 192.168.10.213
- **Adresse IP PVE-03** : 192.168.10.84
- **RAM par nœud** : 64 GB
- **Disques par nœud** : 2×300 GB SAS
- **Réseau inter-nœuds** : Réseau physique Sonatrach (VLAN OVS)

II.7 Conclusion

Ce chapitre a permis de poser les bases théoriques de notre projet. Nous avons défini le concept d'hyperconvergence, comparé les principales solutions open source disponibles et justifié le choix de la stack Proxmox VE + Ceph + Open vSwitch. Nous avons également présenté le rôle de chaque composant dans l'architecture et décrit le modèle de déploiement physique distribué retenu sur trois serveurs Dell PowerEdge R720. Le chapitre suivant présentera la conception détaillée de l'architecture proposée.

Chapitre III

Présentation du Contexte

III.1 Introduction

Ce chapitre présente la conception détaillée de l'infrastructure Cloud privée hyperconvergée proposée. Sur la base des technologies retenues dans le chapitre précédent — Proxmox VE, Ceph et Open vSwitch — nous décrivons ici les choix architecturaux guidant la construction de chaque couche de l'infrastructure. Nous présentons également la conception du mécanisme d'équilibrage dynamique de charge CLBS, ainsi que les outils d'automatisation retenus pour le déploiement et le provisionnement.

L'objectif de ce chapitre est de fournir une vision complète et cohérente de l'architecture avant toute phase d'implémentation, en justifiant chaque décision par des contraintes techniques, des recommandations officielles ou des exigences fonctionnelles identifiées précédemment.

III.2 Vue Globale de l'Architecture

L'architecture proposée repose sur un modèle à trois couches superposées et interdépendantes, chacune apportant une dimension essentielle à l'infrastructure hyperconvergée.

La première couche est la **couche physique**, constituée de trois serveurs Dell PowerEdge R720 mis à disposition par le Centre d'innovation de Sonatrach. Proxmox VE est installé directement en mode bare metal sur chacun de ces serveurs, sans couche de virtualisation intermédiaire, ce qui garantit des performances maximales.

La deuxième couche est la **couche de virtualisation et de cluster**, composée des trois nœuds Proxmox (PVE-01, PVE-02, PVE-03) interconnectés via le réseau Sonatrach. Ces nœuds forment un cluster haute disponibilité géré par Corosync. Chaque nœud exécute simultanément les daemons Ceph pour le stockage distribué et une instance Open vSwitch avec des tunnels VxLAN pour la segmentation réseau. Cette convergence des rôles est l'expression directe du principe HCI.

La troisième couche est la **couche applicative et de services**, hébergeant la VM OPNsense pour le routage et le pare-feu, les VMs de démonstration Sona-Web, le mécanisme CLBS, la stack de monitoring, et les outils d'automatisation Ansible.

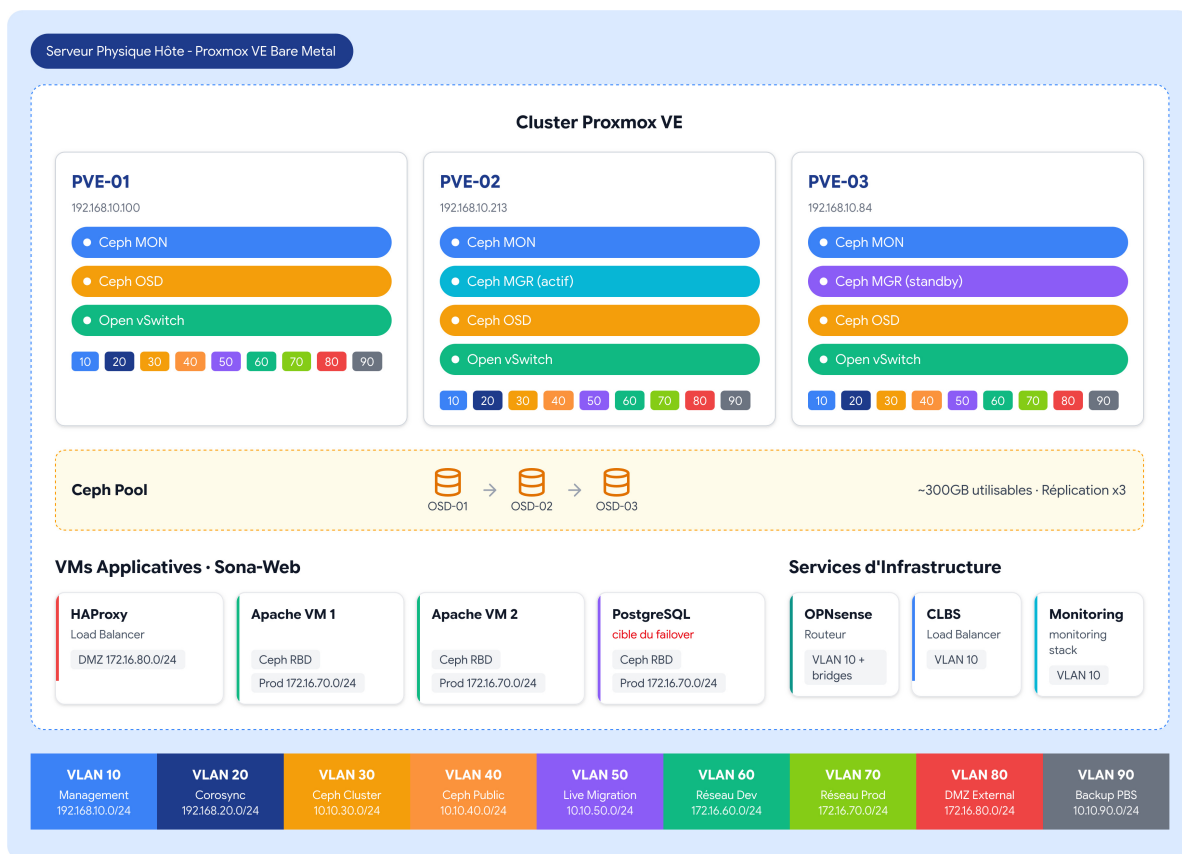


FIGURE III.1 – Architecture Globale de l'Infrastructure HCI

III.3 Partie I : Calcul

III.3.1 Infrastructure Physique

L'infrastructure physique est constituée de trois serveurs Dell PowerEdge R720. Ces serveurs forment les nœuds du cluster Proxmox : PVE-01, PVE-02 et PVE-03. Proxmox VE est installé directement en mode bare metal sur chacun d'eux, sans couche de virtualisation intermédiaire. Le Dell PowerEdge R720 est un serveur rack 2U de la gamme Enterprise de Dell, équipé de processeurs Intel Xeon E5-2600, adapté aux charges de virtualisation intensive et au stockage distribué.

Chaque serveur dispose de deux disques SAS répartis selon leurs rôles respectifs, conformément aux recommandations Proxmox concernant la séparation des entrées/sorties système et de stockage :

- **Disque système (sda — 1 × 300 GB SAS)** : dédié au système d'exploitation Proxmox VE. Cette séparation garantit que les opérations de réplication Ceph ne perturbent pas les I/O système du nœud.
- **Disque OSD Ceph (sdb — 1 × 300 GB SAS)** : affecté à un OSD Ceph, contribuant au stockage distribué du cluster.

TABLE III.1 – Dimensionnement physique des serveurs Dell PowerEdge R720

Composant	Par nœud	Total cluster	Justification
Modèle serveur	Dell R720	3 serveurs	Fourni par Sonatrach
RAM	64 GB	192 GB	Proxmox + Ceph + VMs
Disque OS (<i>sda</i>)	1 × 300 GB SAS	3 × 300 GB	Isolation I/O hôte / Ceph
Disque OSD (<i>sdb</i>)	1 × 300 GB SAS	3 × 300 GB	1 OSD par nœud
IP Management	Tableau 3-2		Réseau Sonatrach 192.168.10.0/24

III.3.2 Architecture du Cluster Proxmox

III.3.2.1 Organisation du Cluster

Le cluster Proxmox est constitué de trois nœuds physiques identiques, interconnectés via le réseau Sonatrach. Le trafic Corosync est transporté sur un réseau logique dédié (VxLAN ID 20) afin de garantir la stabilité du quorum indépendamment des autres flux. Cette organisation à trois nœuds est le minimum requis pour garantir un quorum stable et éviter les situations de split-brain.

TABLE III.2 – Adresses IP de management des nœuds Proxmox

Nœud	Nom d'hôte	Adresse IP Management
Nœud 1	PVE-01	192.168.10.100
Nœud 2	PVE-02	192.168.10.213
Nœud 3	PVE-03	192.168.10.84

III.3.2.2 Mécanisme de Quorum et Corosync

Corosync est le framework de communication utilisé par Proxmox pour assurer le consensus entre les nœuds. Il envoie des messages de heartbeat à intervalles réguliers via le VxLAN 20. Le quorum dans un cluster de trois nœuds est atteint dès que deux nœuds sur trois sont opérationnels. La perte simultanée de deux nœuds entraîne le gel du cluster pour prévenir toute corruption des données.

III.3.2.3 Haute Disponibilité

Le mécanisme de haute disponibilité de Proxmox s'appuie sur Corosync pour la détection des pannes et sur le Resource Manager HA pour les décisions de basculement. Lorsqu'un nœud est déclaré défaillant, le HA Manager identifie les VMs concernées et déclenche leur redémarrage automatique sur un nœud survivant. Ce mécanisme est rendu possible par le stockage partagé Ceph : les disques des VMs étant accessibles depuis n'importe quel nœud, le redémarrage ne nécessite aucun transfert de données préalable. Le délai de basculement est typiquement compris entre 60 et 120 secondes.

Toutes les VMs de l'application Sona-Web sont configurées en mode HA, avec une priorité de basculement définie : HAProxy en premier, suivi des serveurs Flask, puis de la base

PostgreSQL.

III.3.2.4 Matrice des Scénarios de Défaillance

TABLE III.3 – Matrice des scénarios de défaillance et comportements attendus

Scénario	Composant affecté	Comportement	RTO
Panne d'un nœud physique	PVE-01, 02 ou 03	Corosync détecte la perte. HA redémarre les VMs sur les survivants. Ceph passe en degraded mais reste accessible.	60–120 s
Panne d'un OSD	Disque sdb	Ceph marque l'OSD down . Le pool passe en degraded (2/3 répliques). Backfill automatique après rétablissement.	Aucune coupure
Panne du MGR actif	PVE-02 (MGR)	Le MGR standby (PVE-03) prend le relais. Endpoint Prometheus disponible sans interruption.	< 30 s
Panne de deux nœuds	Deux des trois nœuds	Perte du quorum. Le nœud survivant se gèle. Intervention manuelle requise.	N/A
Panne VM OPNsense	VM routeur/pare-feu	Proxmox HA redémarre la VM sur un nœud survivant. Le trafic inter-réseaux est interrompu pendant la durée du redémarrage.	60–120 s
Split-brain	Réseau Vx-LAN 20	Le groupe minoritaire perd le quorum et se gèle. Le groupe majoritaire continue d'opérer.	Aucune coupure côté majoritaire
Panne CLBS	VM CLBS	Aucun impact applicatif. Le service redémarre automatiquement (Restart=always systemd).	Aucune coupure

III.3.3 Mécanisme d'Équilibrage Dynamique : CLBS

III.3.3.1 Contexte et Besoin

Proxmox VE intègre un mécanisme de haute disponibilité réactif : lorsqu'un nœud tombe en panne, les VMs sont redémarrées sur les nœuds survivants. Cependant, Proxmox ne dispose d'aucun mécanisme proactif d'équilibrage de charge, contrairement à VMware vSphere qui propose le DRS (Distributed Resource Scheduler). Sans équilibrage proactif, un nœud peut être saturé à 90% de CPU tandis qu'un autre reste à 20%, dégradant les performances et gaspillant les ressources disponibles.

III.3.3.2 Base Algorithmique : CSLB

Notre mécanisme, nommé **CLBS (Central Load Balancing Scheduler)**, est une adaptation du **CSLB (Central Scheduler Load Balancing)**. L'algorithme original repose sur cinq

phases : évaluation de charge, détermination de rentabilité, calcul du vecteur de transfert, sélection de VM, et exécution de la migration. La charge CPU de chaque nœud est évaluée par rapport à un seuil dynamique (moyenne des charges de tous les nœuds) :

$$\text{seuil} = \frac{\sum_{i=1}^n \alpha_i}{n}, \quad \text{diff} = \left| \text{seuil} - \frac{\alpha_{\min} + \alpha_{\max}}{2} \right|$$

La bande modérée est définie à $\pm \max(\text{diff}, 10\%)$ autour du seuil. Les nœuds hors de cette bande sont candidats à une action d'équilibrage.

III.3.3.3 Modifications Apportées au CSLB

Notre adaptation introduit plusieurs modifications importantes par rapport à l'algorithme original.

- **Implémentation en Go** : retenu pour sa faible empreinte mémoire, sa gestion native de la concurrence via les goroutines, et la simplicité du déploiement sous forme de binaire statique dans un service systemd.
- **Extension à la RAM** : le CSLB original ne considère que le CPU. Notre implémentation calcule une bande modérée indépendante pour la RAM. Un nœud est surchargé si l'une ou l'autre dépasse sa bande respective.
- **Seuil minimum de migration** : une migration n'est déclenchée que si la charge dépasse un seuil absolu de 50% (CPU ou RAM), évitant des déplacements inutiles dans un cluster globalement sous-chargé.
- **Sélection du nœud destination pondérée** : le choix du nœud de destination se fonde sur un score pondéré :

$$\text{score} = \Delta_{CPU} \times 0.7 + \Delta_{RAM} \times 0.3$$

Avant validation, CLBS simule l'état post-migration du nœud cible pour s'assurer que l'ajout de la VM ne le fait pas dépasser la bande haute.

- **Exclusion des VMs système** : OPNsense, la VM monitoring et la VM CLBS sont explicitement exclues du périmètre de migration.
- **Cadence et unicité** : CLBS s'exécute toutes les 60 secondes et ne peut déclencher qu'une seule migration par cycle.
- **Cooldown post-migration** : une période de repos de 5 minutes est appliquée après chaque migration, le temps que les métriques InfluxDB intègrent l'impact du changement.
- **Mécanismes anti-oscillation** : si une même VM est migrée deux fois en moins de 10 minutes, elle est gelée 30 minutes et l'administrateur est notifié. Si 6 migrations ou plus sont enregistrées en 30 minutes, le service se suspend automatiquement pendant une heure (circuit breaker).
- **Collecte des métriques** : CLBS combine l'API REST Proxmox pour les métriques courantes, et InfluxDB via le client Go officiel pour les moyennes glissantes sur 5 minutes, garantissant des décisions fondées sur des données lissées plutôt que sur des pics instantanés.

III.3.3.4 Algorithme de Décision (Pseudocode)

```
toutes les 60 secondes :  
  recuperer metriques CPU et RAM moyenees (5 min) depuis InfluxDB  
  calculer seuil_cpu, bande_moderee_cpu  
  calculer seuil_ram, bande_moderee_ram  
  
pour chaque noeud N :  
  si (cpu_N > bande_haute_cpu OU ram_N > bande_haute_ram)  
    ET (cpu_N > 50% OU ram_N > 50%) :  
  
    candidats_vm = VMs sur N non migrees recemment (cooldown 10  
      min)  
    vm_cible = VM dont la charge est la plus proche du  
      vecteur de transfert calcule (phase 3 CSLB)  
  
    pour chaque noeud destination D (bande legere) :  
      score_D = (cpu_N - cpu_D) * 0.7 + (ram_N - ram_D) * 0.3  
    noeud_dest = D avec score_D maximal  
  
    si noeud_dest trouve :  
      declencher live migration vm_cible -> noeud_dest  
        via API Proxmox (VxLAN 50)  
      enregistrer migration dans InfluxDB  
      appliquer cooldown anti-oscillation (10 min sur vm_cible)
```

Figure III.2 – Pseudocode de l'algorithme CLBS

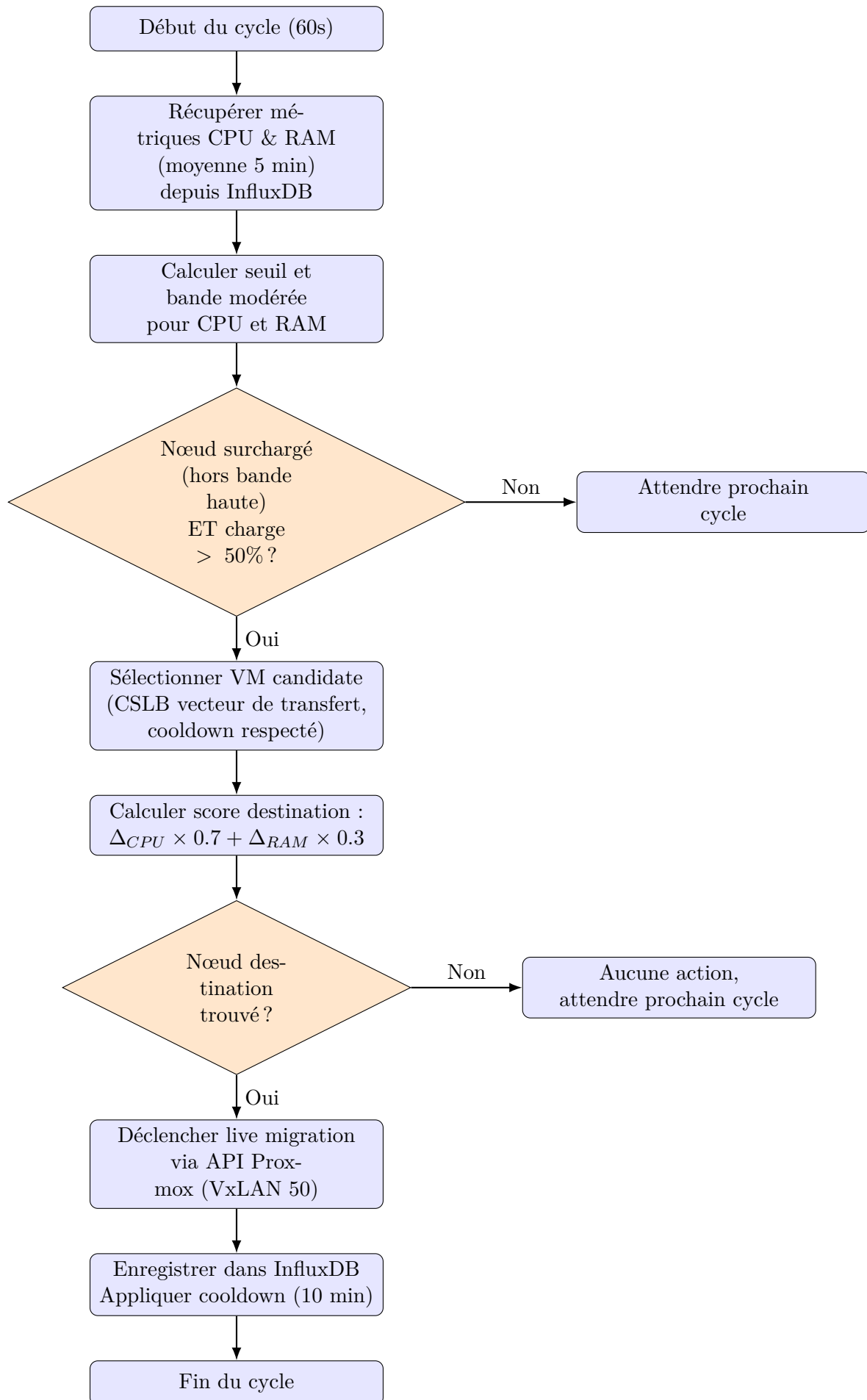


FIGURE III.3 – Logique de Décision du Mécanisme CLBS

III.3.4 Application de Démonstration : Sona-Web

III.3.4.1 Architecture Applicative

Sona-Web est une application web déployée sur le cluster pour démontrer les capacités de haute disponibilité et de live migration. Elle est constituée de quatre VMs Debian 12 interconnectées :

- **HAProxy (VM-201, 192.168.10.111 :8080)** : point d'entrée unique de l'application. Il distribue les requêtes HTTP entre les deux serveurs applicatifs selon l'algorithme round-robin et surveille leur disponibilité toutes les 200 ms (`GET /health`), basculant vers l'instance disponible en moins de 200 ms (`fail 1`) en cas d'indisponibilité.
- **Flask app-1 (VM-202, 192.168.10.112 :8000) et app-2 (VM-203, 192.168.10.114 :8000)** : deux instances identiques de l'application Python Flask. Chaque instance enregistre de manière asynchrone en base de données toutes les requêtes reçues.
- **PostgreSQL (VM-204, 192.168.10.115 :5432)** : base de données centralisée dont le disque est stocké sur un volume Ceph RBD, garantissant un historique cohérent quel que soit le nœud hébergeant les VMs applicatives.

Les quatre VMs sont membres du groupe HA `sona-group` et leurs disques résident sur le pool Ceph RBD partagé. Les adresses IP sont configurées de manière statique pour garantir leur stabilité après migration.

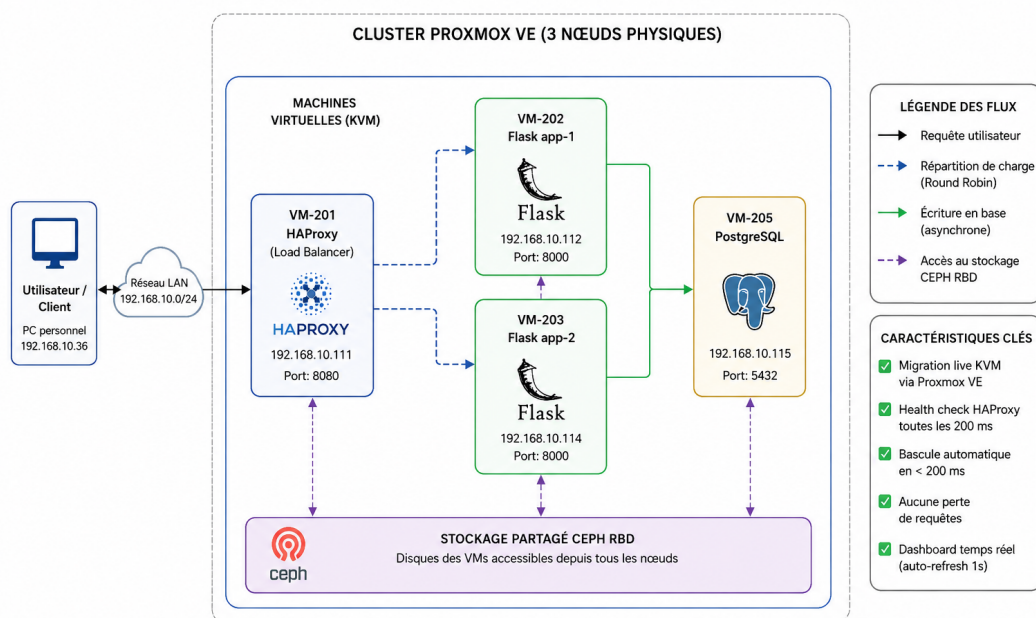


Figure 2 – Architecture alternative de l'application Sona-Web sur cluster Proxmox VE (VM-202 au-dessus de VM-203)

FIGURE III.4 – Architecture Applicative Sona-Web

III.3.4.2 Scénario de Validation HA

Le scénario de validation se déroule en plusieurs temps : affichage de l’alternance round-robin entre app-1 et app-2, puis déclenchement d’une live migration de VM-202 (app-1) vers un autre nœud. Pendant la migration, Proxmox transfère l’état mémoire de la VM sans l’arrêter. Lorsque la synchronisation atteint ~99%, la VM est suspendue quelques dizaines de millisecondes le temps de transférer le delta final, puis reprend sur le nœud cible.

HAProxy détecte l’indisponibilité transitoire en 200 ms maximum et redirige l’intégralité du trafic vers app-2. Une fois la migration terminée, app-1 est remise en service dès le premier health check réussi (rise 1). La requête suivante permet de vérifier la répartition post-migration :

```
SELECT instance_id,
       COUNT(*) AS total
FROM requests
WHERE request_time > NOW() - INTERVAL '1 hour'
GROUP BY instance_id;
```

Figure III.5 – Vérification de la répartition des requêtes en base

III.3.5 Architecture du Monitoring

La stack de monitoring est déployée dans un conteneur LXC dédié à l’adresse 192.168.10.99. Elle comprend trois composants complémentaires couvrant des périmètres de métriques distincts.

InfluxDB reçoit les métriques générales de Proxmox (CPU, RAM, réseau, stockage des nœuds et des VMs) via le mécanisme natif *Metric Server*, sans agent supplémentaire. Cet historique est utilisé à la fois par Grafana pour la visualisation et par le service CLBS pour ses décisions de migration.

Prometheus collecte les métriques détaillées du cluster Ceph via l’endpoint exposé par le nœud MGR actif sur le port 9283. Les métriques applicatives du service CLBS sont également exposées sur le port 9101 de la VM dédiée.

Grafana est connecté à InfluxDB et à Prometheus comme sources de données, permettant de construire des tableaux de bord unifiés couvrant l’ensemble de l’infrastructure : utilisation des ressources par nœud, état du cluster Ceph, historique des migrations CLBS, et alertes de surcharge.

III.3.6 Automatisation

III.3.6.1 Ansible : Périmètre et Playbooks

Ansible est l’outil d’automatisation principal, exécuté depuis les postes de travail des développeurs via SSH sur le réseau de management. Trois playbooks couvrent les principaux besoins d’automatisation.

- **Playbook 1 — Intégration d'un nouveau nœud** : automatise l'ensemble de la procédure d'intégration d'un serveur fraîchement installé (via PXE) dans le cluster existant. Il couvre la configuration réseau OVS/VxLAN, l'adhésion au cluster Corosync, l'initialisation de l'OSD Ceph, et la configuration des exports de métriques vers InfluxDB.
- **Playbook 2 — Provisionnement de VMs** : automatise la création de VMs et de conteneurs LXC à partir de templates cloud-init stockés dans le pool Ceph. Il intègre la configuration réseau via DHCP OPNsense et l'assignation au bon bridge OVS selon le réseau cible (Dev, Prod ou DMZ).
- **Playbook 3 — Migration Dev vers Prod/DMZ** : gère la promotion d'une VM ou d'un ensemble de VMs vers la production ou la DMZ, en reconfigurant les interfaces réseau, les règles de pare-feu OPNsense et en vérifiant la connectivité post-migration.

III.3.6.2 Pipeline PXE/TFTP

Dans une optique de scalabilité, un pipeline de provisionnement automatique en deux phases a été mis en place pour intégrer tout nouveau serveur physique sans intervention manuelle.

Phase 1 — Installation automatique via PXE. Lorsqu'un nouveau serveur est mis sous tension, la séquence suivante s'exécute automatiquement :

1. Le serveur envoie une requête DHCP ; OPNsense lui attribue une adresse temporaire et lui indique l'adresse du serveur de boot ainsi que le fichier bootloader à récupérer (options DHCP 66 et 67).
2. Le firmware PXE contacte le serveur TFTP (UDP/69) pour télécharger le bootloader (`pxelinux.0`) et le kernel Linux d'amorçage. TFTP est le seul protocole utilisable dans un firmware pre-OS.
3. Une fois le kernel démarré en mémoire, l'image d'installation Proxmox VE (~800 Mo) est téléchargée via HTTP depuis un serveur Nginx dédié. HTTP (TCP) est retenu pour ce transfert car TFTP (UDP) est inadapté aux fichiers volumineux.
4. L'installateur Proxmox lit un fichier de réponses (*Answer File*) hébergé sur le serveur HTTP, contenant le partitionnement, le nom d'hôte, le fuseau horaire et la langue. L'installation s'effectue sans aucune frappe clavier.
5. Le serveur redémarre automatiquement sur Proxmox VE fraîchement installé.

Phase 2 — Intégration au cluster via Ansible. À l'issue du redémarrage, le nœud dispose de Proxmox VE fonctionnel mais isolé. Un playbook Ansible prend le relais via SSH pour réaliser l'intégration complète :

- Adhésion au cluster **Corosync** ;
- Configuration des interfaces **Open vSwitch** et des tunnels **VxLAN** ;
- Intégration des disques physiques comme **OSDs** dans le cluster **Ceph** ;

- Rattachement au groupe HA Proxmox (`sona-ha-group`) ;
- Déclaration du nœud comme cible dans **Prometheus**.

Ce découpage en deux phases répond à une contrainte fondamentale : PXE et TFTP interviennent dans un environnement pre-OS où aucun client SSH ne peut fonctionner, tandis qu'Ansible nécessite un système d'exploitation opérationnel. L'ensemble du pipeline permet l'ajout d'un nouveau nœud en moins de 25 minutes, sans intervention manuelle.

III.4 Partie II : Stockage

III.4.1 Architecture du Cluster Ceph

III.4.1.1 Composants du Cluster

Le cluster Ceph est composé des services suivants, répartis sur les trois nœuds physiques :

- **Monitors (MON)** : un monitor par nœud, soit trois au total. Les monitors maintiennent la carte du cluster (*cluster map*) et assurent le consensus distribué. Trois monitors garantissent un quorum stable (2/3 suffisants).
- **Managers (MGR)** : PVE-02 héberge le MGR actif, PVE-03 le MGR en standby. Le MGR expose les métriques du cluster via un endpoint REST utilisé par Prometheus.
- **OSDs (Object Storage Daemons)** : un OSD par nœud sur le disque `sdb` (300 GB SAS), soit trois OSDs contribuant au pool de stockage distribué.

III.4.1.2 Dimensionnement du Pool

Avec un facteur de réplication de 3, la capacité utilisable est calculée ainsi :

$$\text{Capacité utilisable} = \frac{3 \times 300 \text{ GB}}{3} = 300 \text{ GB}$$

L'algorithme CRUSH est configuré pour respecter les domaines de défaillance au niveau du nœud physique, garantissant que les trois répliques d'un même bloc ne se retrouvent jamais sur le même serveur. Le tableau suivant présente la répartition prévue :

TABLE III.4 – Répartition du stockage Ceph par usage

Usage	Stockage alloué
VM OPNsense	20 GB
HAProxy VM	16 GB
Flask VM 1	32 GB
Flask VM 2	32 GB
PostgreSQL VM	100 GB
VM Monitoring (InfluxDB + Prometheus + Grafana)	100 GB
VM CLBS	32 GB
Marge, snapshots et templates	~200 GB
Total estimé	~532 GB (thin)

Le total estimé dépasse la capacité brute de 300 GB car le thin provisioning de Ceph RBD ne consomme l'espace que proportionnellement aux données réellement écrites.

III.4.1.3 Configuration CRUSH et Domaines de Défaillance

Lors de la création du cluster Ceph via Proxmox VE, une règle CRUSH par défaut est générée automatiquement, organisant les OSDs selon la hiérarchie `root > host > osd` avec la politique `chooseleaf firstn 0 type host`. Cela garantit que les trois répliques d'un objet sont placées sur trois hôtes physiques distincts.

TABLE III.5 – Paramètres CRUSH appliqués automatiquement par Proxmox

Paramètre	Valeur
Domaine de défaillance	<code>host</code> — généré automatiquement par Proxmox
Facteur de réplication (<code>size</code>)	3 — une réplique par nœud physique
<code>min_size</code>	2 (défaut Ceph) — pool accessible avec 2 nœuds opérationnels

III.5 Partie III : Réseau

III.5.1 Contexte et Contraintes Réseau

Le réseau Sonatrach met à disposition un sous-réseau `192.168.10.0/24` utilisé pour l'accès SSH et l'interface web Proxmox. La connectivité Internet des nœuds est assurée par un poste jouant le rôle de routeur externe à l'adresse `192.168.10.36`, qui réalise du NAT et du forwarding IP.

Le switch Sonatrach n'autorise que le VLAN 10 natif et ne permet pas la configuration de VLANs supplémentaires. Pour créer les réseaux logiques nécessaires à l'infrastructure, **VxLAN (Virtual Extensible LAN)** a été retenu comme technologie d'encapsulation. VxLAN permet de créer des réseaux logiques de niveau 2 tunnelisés au-dessus du réseau IP existant, sans modifier la configuration du switch. Open vSwitch gère la création et la terminaison de ces tunnels sur chaque nœud Proxmox.

III.5.2 Topologie Réseau

III.5.2.1 Deux Niveaux de Segmentation

La topologie retenue s'articule autour de deux niveaux complémentaires.

Niveau 1 — Réseaux d'infrastructure (VxLAN inter-nœuds). Pour les réseaux de contrôle (Corosync, Ceph Cluster, Ceph Public, Live Migration), des tunnels VxLAN point-à-point en maillage complet sont établis entre les trois nœuds. Chaque nœud crée deux ports VxLAN vers chacun des deux autres, garantissant une connectivité directe sans point de transit central.

Niveau 2 — Réseaux applicatifs (OPNsense). Pour les réseaux Dev, Prod et DMZ, une architecture centralisée autour d'une VM OPNsense est utilisée. Trois bridges OVS internes sont configurés sur chaque nœud. OPNsense dispose de quatre interfaces virtuelles : WAN connectée au réseau de management (192.168.10.65, gateway 192.168.10.36), et trois interfaces LAN connectées aux bridges Dev, Prod et DMZ.

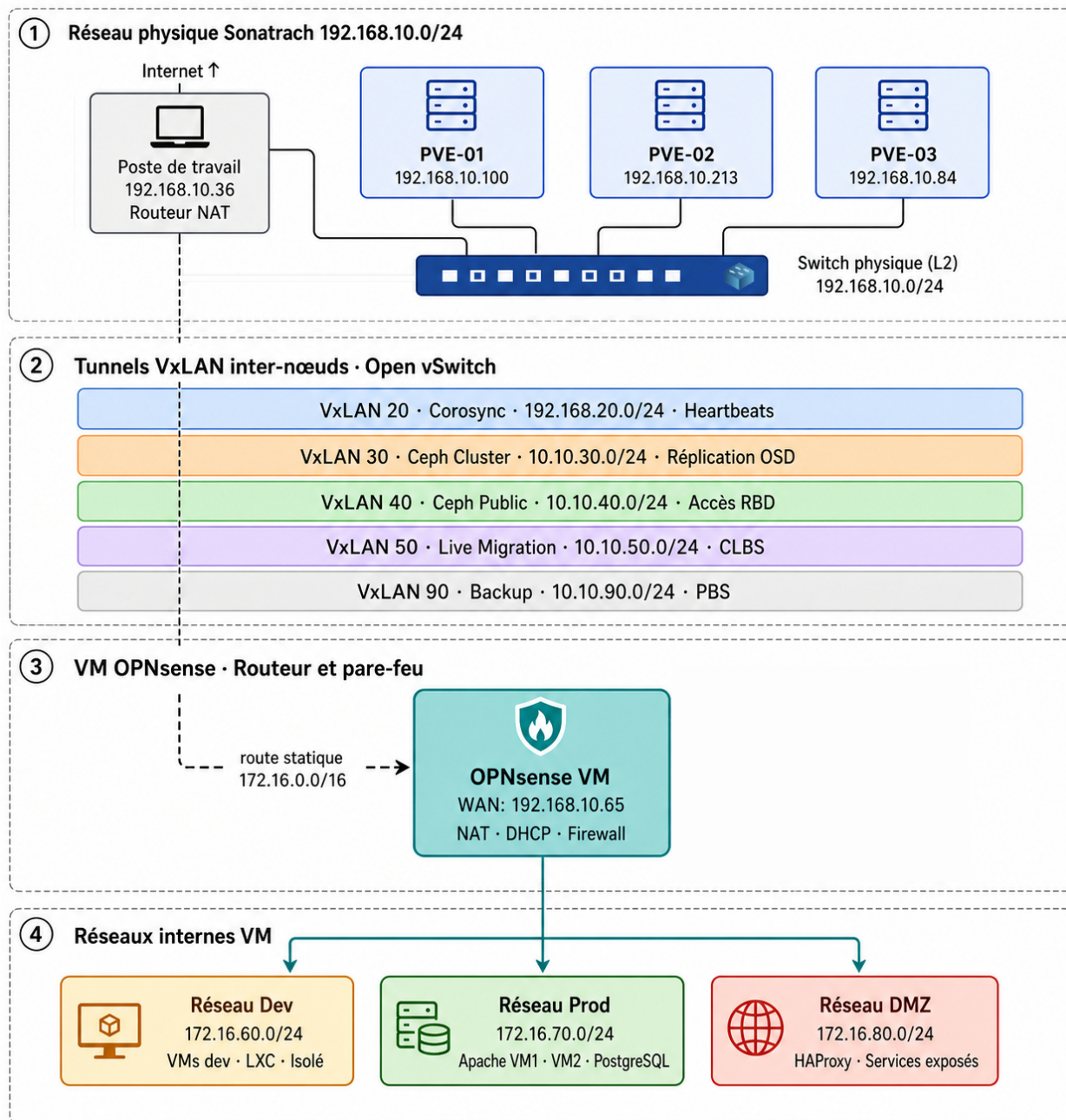


FIGURE III.6 – Topologie Réseau et Plan VxLAN

III.5.2.2 Bridges Open vSwitch par Nœud

TABLE III.6 – Bridges OVS et tunnels VxLAN par nœud Proxmox

Bridge OVS	VxLAN associé	Sous-réseau	Rôle
vmbr0	Réseau physique natif	192.168.10.0/24	Bridge principal. Accès management, SSH, Web UI Proxmox, interface WAN d'OPNsense.
vmbr60	vxlan60-int	172.16.60.0/24	Réseau Dev. OPNsense y expose son interface LAN1.
vmbr70	vxlan70-int	172.16.70.0/24	Réseau Prod. OPNsense y expose son interface LAN2.
vmbr80	vxlan80-int	172.16.80.0/24	Réseau DMZ. OPNsense y expose son interface LAN3.

III.5.3 Plan d'Adressage

Le plan d'adressage est organisé en deux familles : les réseaux VxLAN d'infrastructure pour les flux de contrôle, et les réseaux applicatifs internes gérés par OPNsense.

Réseaux VxLAN d'infrastructure :

- **VxLAN 10 — Management (192.168.10.0/24)** : Web UI Proxmox, SSH, API (réseau physique existant).
- **VxLAN 20 — Corosync (192.168.20.0/24)** : Heartbeats et quorum cluster.
- **VxLAN 30 — Ceph Cluster (10.10.30.0/24)** : Réplication OSD interne.
- **VxLAN 40 — Ceph Public (10.10.40.0/24)** : Accès des VMs au stockage RBD.
- **VxLAN 50 — Live Migration (10.10.50.0/24)** : Migrations à chaud et CLBS.

Réseaux applicatifs internes :

- **Réseau Dev (172.16.60.0/24, gateway 172.16.60.1)** : environnements de développement et de test.
- **Réseau Prod (172.16.70.0/24, gateway 172.16.70.1)** : environnements de production.
- **Réseau DMZ (172.16.80.0/24, gateway 172.16.80.1)** : services exposés, dont HAProxy.

TABLE III.7 – Adressage IP statique des nœuds Proxmox par réseau logique

Nœud	Management	VxLAN 20	VxLAN 30	VxLAN 40	VxLAN 50
PVE-01	192.168.10.100	192.168.20.11	10.10.30.11	10.10.40.11	10.10.50.11
PVE-02	192.168.10.213	192.168.20.12	10.10.30.12	10.10.40.12	10.10.50.12
PVE-03	192.168.10.84	192.168.20.13	10.10.30.13	10.10.40.13	10.10.50.13

III.5.4 VM OPNsense : Routeur et Pare-feu Centralisé

III.5.4.1 Interfaces et Rôles

La VM OPNsense constitue le cœur du réseau applicatif. Il s'agit d'une distribution FreeBSD dédiée aux fonctions de routage et de sécurité réseau. Dans cette architecture, OPNsense est déployée avec quatre interfaces réseau virtuelles :

- **Interface WAN (192.168.10.65/24)** : connectée au bridge OVS de management, avec gateway 192.168.10.36. Assure l'accès Internet sortant via NAT pour toutes les VMs internes.
- **Interface LAN1 (172.16.60.1/24)** : passerelle du réseau Dev.
- **Interface LAN2 (172.16.70.1/24)** : passerelle du réseau Prod.
- **Interface LAN3 (172.16.80.1/24)** : passerelle du réseau DMZ.

OPNsense intègre nativement dnsmasq, utilisé comme serveur DHCP pour les trois réseaux internes. L'attribution des adresses IP aux VMs est ainsi automatisée dès leur connexion à l'un des bridges applicatifs, simplifiant le provisionnement via les playbooks Ansible.

III.5.4.2 Haute Disponibilité d'OPNsense

OPNsense étant le point central du routage et du filtrage, sa défaillance isolerait l'ensemble des réseaux internes. Sa résilience est assurée par son intégration au groupe HA Proxmox. En cas de défaillance du nœud hébergeant la VM, Corosync la détecte et le HA Manager déclenche son redémarrage sur un nœud survivant, directement depuis le volume Ceph RBD. Le délai de basculement est de 60 à 120 secondes.

Pour une architecture de production nécessitant un basculement transparent, une configuration active/passive avec le protocole CARP (natif dans OPNsense) réduirait ce délai à moins de 5 secondes.

III.5.5 Architecture du Pare-feu

III.5.5.1 Pare-feu OPNsense : Filtrage Inter-réseaux

OPNsense applique les politiques de filtrage entre les réseaux applicatifs Dev, Prod et DMZ. Les règles de conception retenues sont les suivantes :

Règles autorisées :

- Dev → WAN : autorisé pour les mises à jour et les dépôts de paquets.
- DMZ → WAN : autorisé pour les services exposés nécessitant un accès sortant.
- Prod → DMZ : autorisé pour la communication entre production et services exposés (HAProxy).

Règles bloquées :

- Dev → Prod : bloqué pour isoler les environnements de développement de la production.
- Dev → DMZ : bloqué pour empêcher l'accès aux services exposés depuis le développement.
- Prod → WAN direct : bloqué, le trafic de production passe obligatoirement par la DMZ.

III.5.5.2 Pare-feu Proxmox : Filtrage au Niveau Hyperviseur

Proxmox VE intègre nativement un mécanisme de pare-feu basé sur iptables, configurable aux niveaux cluster, nœud et VM individuelle. Les règles de conception retenues pour protéger la plateforme sont les suivantes :

- **Isolation des réseaux Ceph** : interdire tout trafic des réseaux applicatifs (172.16.x.0/24) vers les réseaux Ceph Cluster (VxLAN 30) et Ceph Public (VxLAN 40).
- **Isolation du réseau Corosync** : interdire tout accès extérieur au VxLAN 20. Seuls les nœuds Proxmox communiquent sur ce réseau.
- **Protection du management** : restreindre l'accès à l'interface web Proxmox (port 8006) et SSH (port 22) aux seules adresses du sous-réseau 192.168.10.0/24.
- **Moindre privilège par VM** : n'autoriser que les ports applicatifs strictement nécessaires sur chaque VM.

Architecture du pare-feu : deux niveaux de filtrage

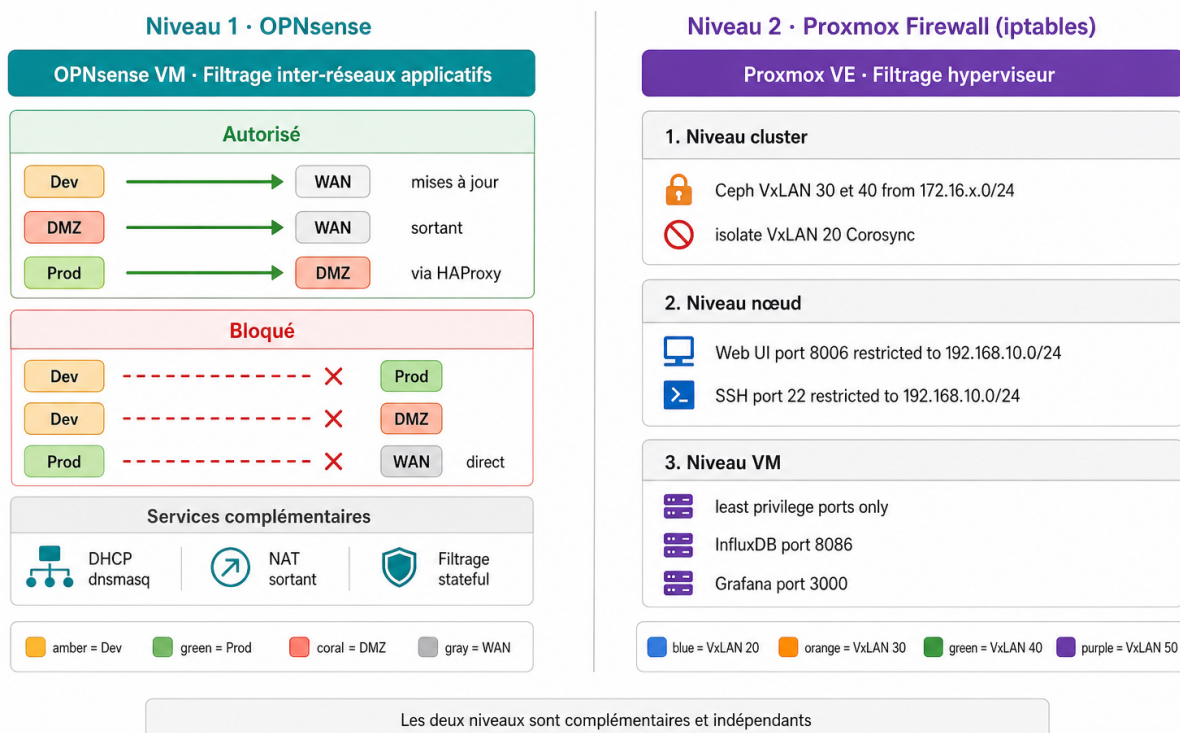


FIGURE III.7 – Architecture du Pare-feu : OPNsense et Proxmox Firewall

III.6 Conclusion

Ce chapitre a présenté la conception complète de l'infrastructure Cloud privée hyperconvergée, organisée autour des trois piliers fondamentaux de l'HCI. La partie Calcul couvre les serveurs physiques, le cluster Proxmox avec Corosync et HA, le mécanisme CLBS, l'application Sona-Web, la stack de monitoring et l'automatisation via Ansible et PXE. La partie Stockage détaille le cluster Ceph avec ses composants MON, MGR et OSD, le dimensionnement du pool et la politique CRUSH. La partie Réseau expose l'architecture VxLAN gérée par Open vSwitch, le plan d'adressage complet, le routage centralisé via OPNsense et le plan de pare-feu à deux niveaux.

L'ensemble de ces choix de conception constitue une base cohérente et documentée pour la phase d'implémentation présentée dans le chapitre suivant.

Chapitre IV

Implémentation de la Solution Proposée

IV.1 Introduction

Ce chapitre présente la mise en œuvre concrète de la solution proposée dans le chapitre précédent. L'objectif est de montrer les étapes réelles de déploiement de l'infrastructure hyperconvergée, depuis l'installation de la plateforme de virtualisation jusqu'aux services applicatifs et aux mécanismes d'automatisation. Pour améliorer la lisibilité, l'implémentation est organisée en trois grandes parties : la couche calcul, la couche stockage, puis la couche réseau.

Les opérations ont été réalisées avec plusieurs outils complémentaires : l'interface web Proxmox VE, la ligne de commande Linux, les fichiers de configuration système, OPNsense, Ceph, ainsi que des playbooks Ansible pour automatiser certaines tâches répétitives. Des captures d'écran sont prévues aux points importants afin d'illustrer visuellement les résultats obtenus.

IV.2 Partie I — Couche Calcul

IV.2.1 Préparation de l'Infrastructure Physique

IV.2.1.1 Installation de Proxmox VE sur les nœuds

L'installation de la plateforme de virtualisation a commencé par le téléchargement de l'image ISO de Proxmox VE version 9.1.1. Cette image a ensuite été copiée sur une clé USB bootable utilisée pour démarrer chaque serveur Dell PowerEdge R720. Une installation séparée a été réalisée sur chacun des trois nœuds du cluster.

Pendant l'installation, le disque système `sda` a été utilisé pour héberger Proxmox VE. Une configuration réseau statique a aussi été appliquée afin de donner à chaque nœud une adresse de management fixe sur le réseau physique `192.168.10.0/24`. Les adresses retenues sont `192.168.10.100` pour PVE-01, `192.168.10.213` pour PVE-02 et `192.168.10.84` pour PVE-03.

Une fois l'installation terminée, la vérification a été effectuée à travers l'interface web de Proxmox, accessible sur le port `8006`. Cette étape a permis de confirmer que chaque nœud était opérationnel et prêt pour la suite du déploiement.

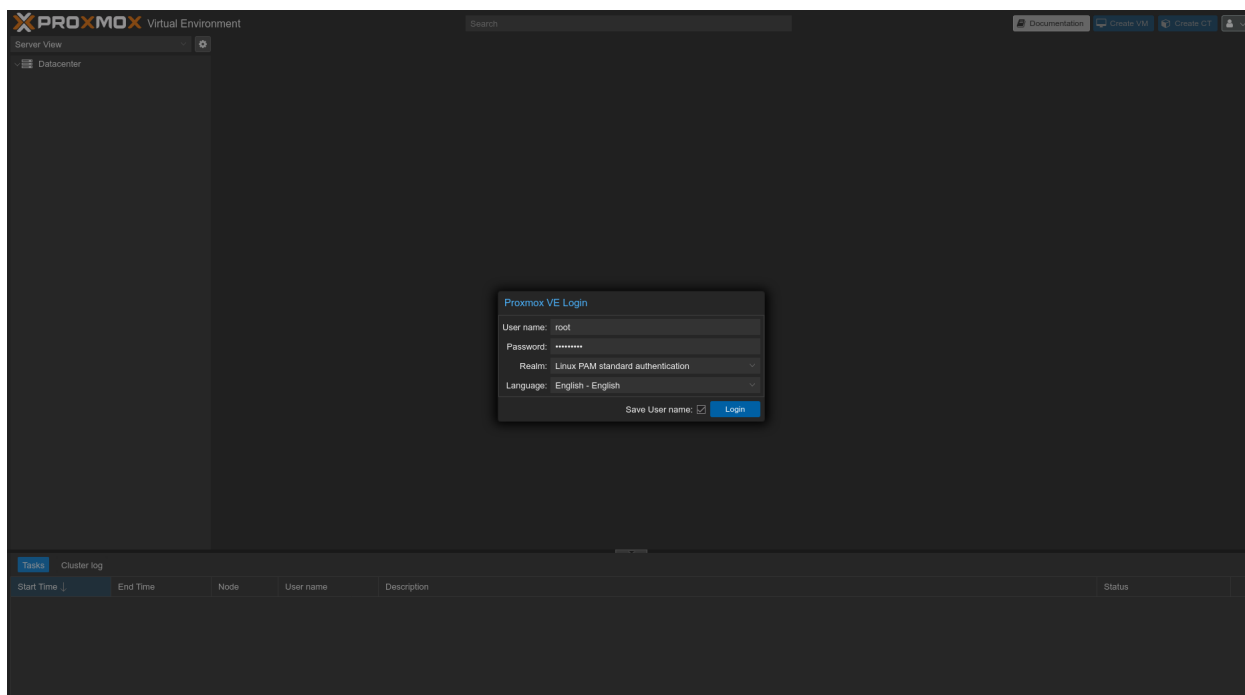


FIGURE IV.1 – Interface web Proxmox au premier login sur PVE-01

IV.2.1.2 Configuration de la connectivité Internet via le laptop gateway

Dans l'environnement de laboratoire, aucun routeur physique dédié n'était disponible pour fournir l'accès Internet aux nœuds Proxmox. Pour cette raison, un laptop a été configuré pour jouer le rôle de passerelle NAT entre le réseau interne de l'infrastructure et Internet. Il possède deux interfaces : `enp0s20f0u4c2` côté infrastructure avec l'adresse `192.168.10.36/24`, et `wlan0` côté Internet avec l'adresse `10.247.78.8/24`.

La première étape consiste à activer le routage IP afin que le système puisse transférer les paquets d'une interface à l'autre. Cette activation doit être faite de manière immédiate puis rendue permanente.

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo vim /etc/sysctl.d/99-ipforward.conf
# Ajouter :
net.ipv4.ip_forward=1
```

Figure IV.2 – Activation du routage IP sur le laptop

Ensuite, des règles `iptables` ont été ajoutées pour réaliser la translation d'adresses NAT et autoriser le forwarding du trafic entre les interfaces du laptop.

```
sudo iptables -t nat -A POSTROUTING -o wlan0 -j MASQUERADE
sudo iptables -A FORWARD -i enp0s20f0u4c2 -o wlan0 -j ACCEPT
sudo iptables -A FORWARD -i wlan0 -o enp0s20f0u4c2 -m state --state
RELATED,ESTABLISHED -j ACCEPT
```

Figure IV.3 – Configuration NAT avec iptables

Pour éviter de perdre ces règles après redémarrage, elles ont été sauvegardées puis activées automatiquement au boot du laptop.

```
sudo pacman -S iptables-nft
sudo iptables-save | sudo tee /etc/iptables/iptables.rules
sudo systemctl enable iptables
```

Figure IV.4 – Persistance des règles iptables

Sur chaque nœud Proxmox, la passerelle par défaut a été définie vers 192.168.10.36, avec des serveurs DNS publics. Cette configuration a été ajoutée dans le fichier `/etc/network/interfaces`.

```
auto vmbr0
iface vmbr0 inet static
    address 192.168.10.XX/24
    gateway 192.168.10.36
    dns-nameservers 8.8.8.8 8.8.4.4
```

Figure IV.5 – Configuration réseau de base d'un nœud Proxmox

Enfin, des routes statiques ont été ajoutées sur le laptop afin de rediriger les réseaux internes 172.16.60.0/24, 172.16.70.0/24 et 172.16.80.0/24 vers la VM OPNsense, qui assure ensuite le routage vers ces sous-réseaux.

```
nmcli connection modify "Wired connection 1" \
+ipv4.routes "172.16.60.0/24 192.168.10.65" \
+ipv4.routes "172.16.70.0/24 192.168.10.65" \
+ipv4.routes "172.16.80.0/24 192.168.10.65"

nmcli connection up "Wired connection 1"
```

Figure IV.6 – Ajout des routes statiques vers OPNsense

Des tests simples comme `ping 8.8.8.8`, `ping google.com` et `apt update` ont confirmé que les nœuds disposaient correctement d'un accès Internet.

IV.2.2 Formation du cluster Proxmox et haute disponibilité

IV.2.2.1 Création et jonction au cluster

Après l'installation de Proxmox VE sur les trois serveurs, la phase suivante a consisté à former un cluster unique. Cette opération a été lancée depuis le premier nœud PVE-01, qui devient le nœud initial du cluster.

```
pvecm create Cluster-Proxmox-VE
```

Figure IV.7 – Création du cluster Proxmox

Les deux autres nœuds ont ensuite rejoint ce cluster en utilisant l'adresse IP du nœud principal.

```
pvecm add 192.168.10.100
```

Figure IV.8 – Jonction de PVE-02 et PVE-03 au cluster

Le réseau Corosync, utilisé pour le heartbeat entre les nœuds, a été isolé sur le VxLAN 20 afin de séparer le trafic de cluster du reste du trafic d'administration ou de production. Une fois la configuration terminée, la commande de vérification a confirmé la présence des trois nœuds avec quorum atteint.

```
pvecm status
```

Figure IV.9 – Vérification de l'état du cluster

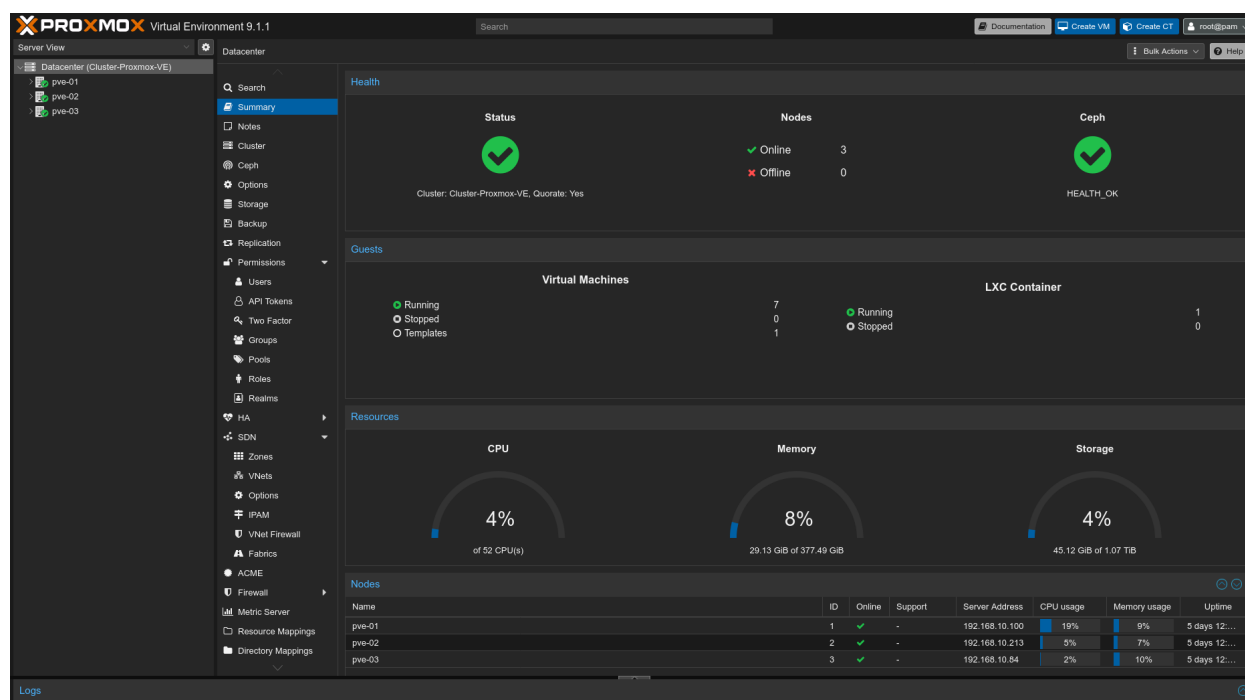


FIGURE IV.10 – Vue du cluster Proxmox montrant les trois nœuds en ligne

IV.2.2.2 Configuration de la migration à chaud

La migration à chaud des machines virtuelles a été configurée pour utiliser le réseau dédié VxLAN 50. Ce choix permet de ne pas mélanger le trafic de migration avec le trafic utilisateur ou le trafic de stockage.

Dans la configuration du datacenter Proxmox, les paramètres de migration ont été ajustés pour utiliser un réseau spécifique et garder un niveau de sécurité acceptable. Une migration manuelle de test a ensuite été réalisée depuis l'interface web afin de vérifier le déplacement transparent d'une VM d'un nœud à un autre sans interruption notable du service.

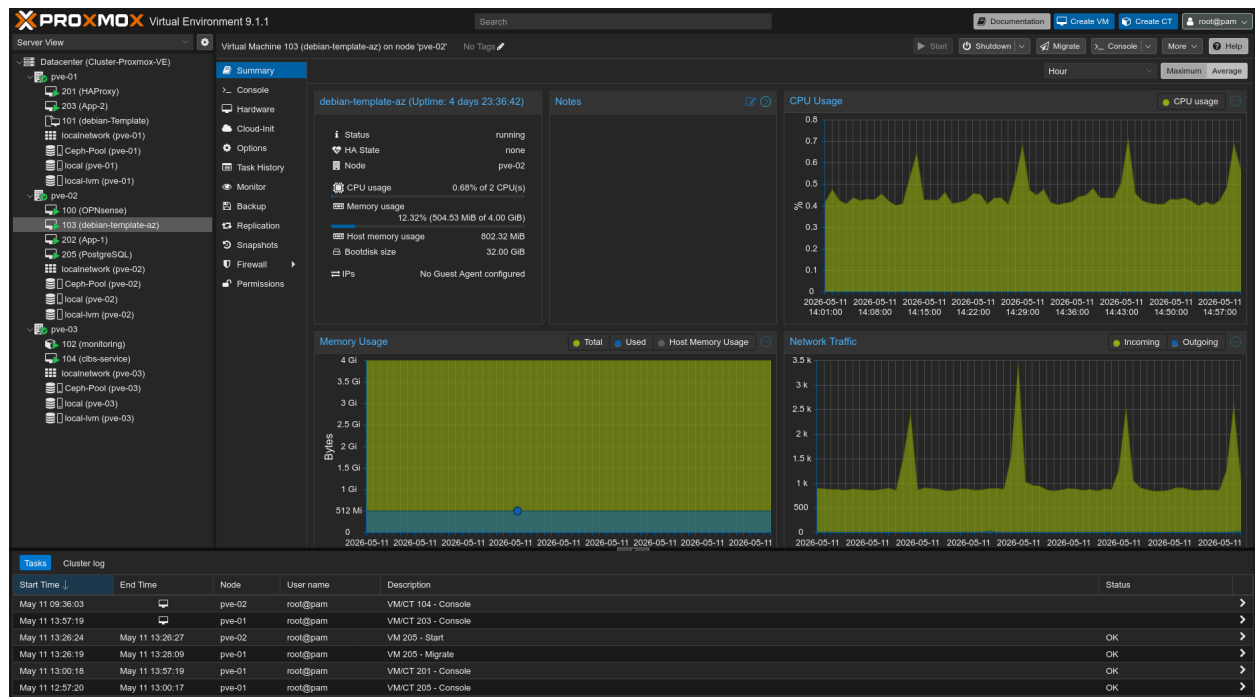


FIGURE IV.11 – Avant la migration live d’une machine virtuelle

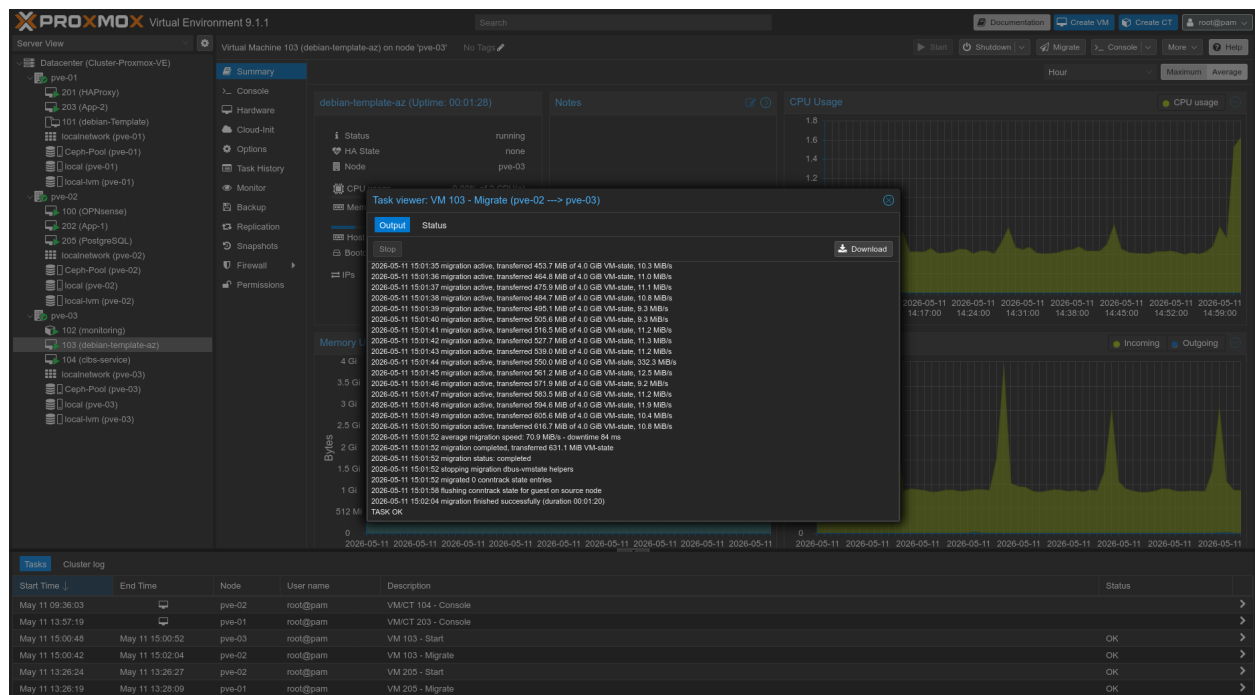


FIGURE IV.12 – Après la migration live d’une machine virtuelle

IV.2.3 Déploiement de la stack de monitoring

Afin de superviser l’état global de l’infrastructure, une stack de monitoring a été déployée dans un conteneur LXC dédié appelé `monitoring`, avec l’adresse `192.168.10.99`. Ce conteneur centralise InfluxDB, Prometheus et Grafana.

IV.2.3.1 Déploiement d’InfluxDB

InfluxDB a été installé pour recevoir les métriques envoyées directement par Proxmox à travers le système natif *Metric Server*. Un bucket nommé **proxmox-metrics** a été créé afin de stocker les mesures CPU, mémoire, réseau et stockage de chaque nœud du cluster.

Une fois la configuration validée dans l’interface web Proxmox, les métriques ont commencé à être transmises automatiquement sans agent supplémentaire. La présence des séries temporelles dans l’interface InfluxDB a confirmé le bon fonctionnement de ce mécanisme.

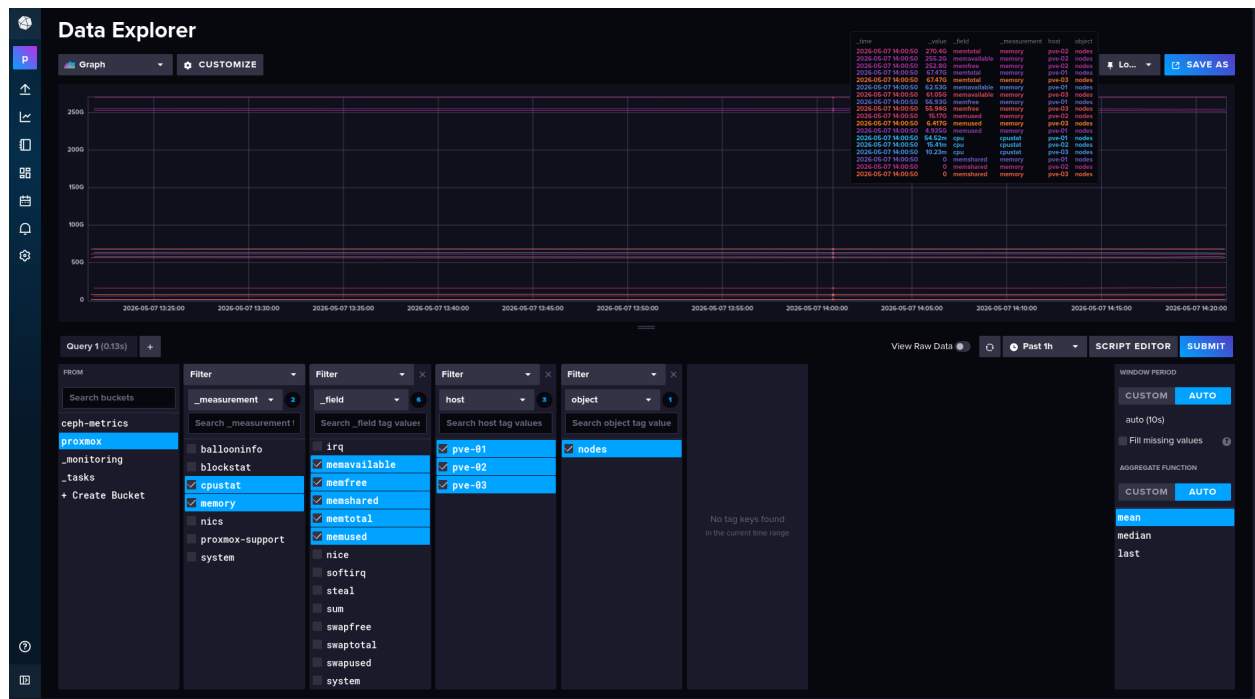


FIGURE IV.13 – Explorateur InfluxDB montrant les métriques reçues depuis Proxmox

IV.2.3.2 Déploiement de Prometheus

Prometheus a été installé dans le même conteneur afin de collecter les métriques exposées par Ceph et par le service applicatif CLBS. La configuration du fichier **prometheus.yml** contient deux jobs principaux : un job pour les métriques Ceph via le module MGR Prometheus, et un job pour l’application CLBS développée en Go.

```
global:
  scrape_interval: 30s
  evaluation_interval: 30s

scrape_configs:
  - job_name: ceph
    static_configs:
      - targets:
        - "192.168.10.213:9283"
        - "192.168.10.83:9283"
        - "192.168.10.100:9283"
    labels:
      cluster: "proxmox-lab"
```

```
- job_name: "clbs"
  static_configs:
    - targets: ["192.168.10.104:9101"]
      scrape_interval: 30s
      scrape_timeout: 10s
```

Figure IV.14 – Extrait du fichier prometheus.yml

L'interface web Prometheus a ensuite permis de confirmer que les différentes cibles apparaissaient avec l'état UP.

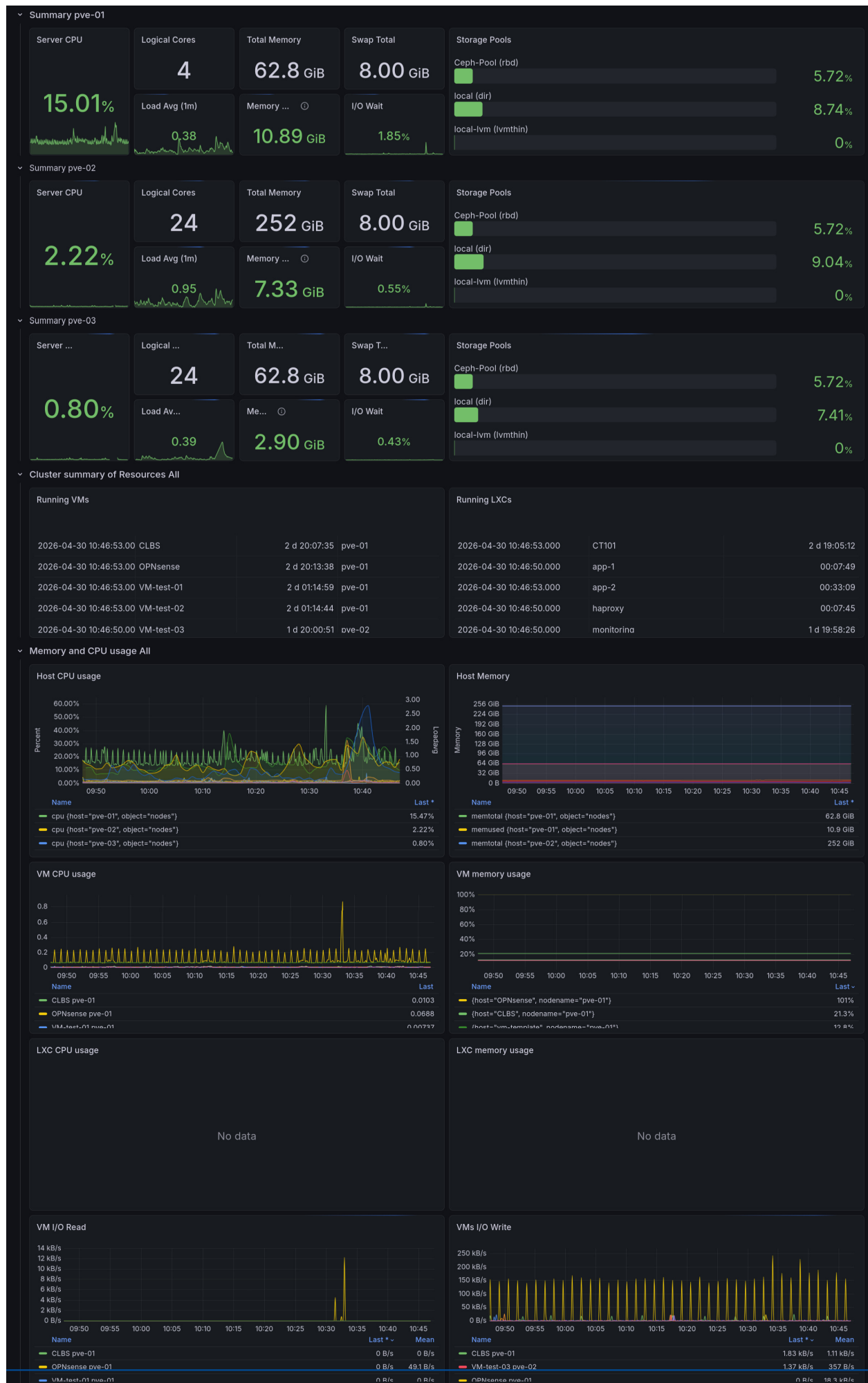
IV.2.3.3 Déploiement de Grafana

Grafana a été ajouté comme couche de visualisation unifiée pour les métriques de Proxmox, Ceph et CLBS. Deux sources de données ont été configurées : InfluxDB pour les métriques de Proxmox, et Prometheus pour les métriques Ceph et applicatives.

TABLE IV.1 – Dashboards Grafana configurés

Dashboard	Source	Métriques principales
Proxmox Cluster	InfluxDB	CPU, RAM, réseau par nœud
Ceph	Prometheus	Santé cluster, pools, latence OSD
CLBS	Prometheus	Historique des migrations

Trois dashboards principaux ont ainsi été utilisés : un dashboard infrastructure Proxmox, un dashboard Ceph, et un dashboard personnalisé pour le service CLBS.



IV.2.4 Implémentation du mécanisme CLBS

IV.2.4.1 Structure du service

Le service CLBS (*Cluster Load Balancing Service*) a été développé en langage Go dans une VM dédiée accessible sur le réseau de management. Son rôle est d'analyser périodiquement la charge CPU du cluster et de déclencher automatiquement des migrations live lorsque cela est nécessaire.

Le code est structuré en plusieurs composants : un lecteur InfluxDB pour les métriques historiques, un client API Proxmox pour piloter le cluster, un calculateur de bandes de charge, et un moteur principal de décision. Les paramètres de connexion sont placés dans un fichier `.env` chargé au démarrage du service.

IV.2.4.2 Logique de décision

À chaque cycle, le service récupère la charge des nœuds, calcule les bornes de charge modérée, puis classe les nœuds dans les catégories *heavy*, *moderate* ou *light*. Si un déséquilibre est détecté entre un nœud surchargé et un nœud léger, le moteur choisit la VM la plus adaptée à migrer.

Plusieurs mécanismes de protection ont été ajoutés pour éviter les migrations excessives, comme un temps de cooldown global, un cooldown individuel par VM et une suspension temporaire en cas de fréquence de migration trop élevée.

IV.2.4.3 Déploiement comme service systemd

Une fois compilé, le binaire Go a été déployé sur la VM CLBS puis intégré comme service `systemd` afin d'assurer un démarrage automatique et une relance en cas de panne.

```
systemctl enable clbs
systemctl start clbs
journalctl -u clbs.service -f -o cat
```

Figure IV.16 – Gestion du service CLBS avec systemd

IV.2.4.4 Validation du mécanisme

Pour valider le comportement du service, deux VMs de test ont été créées puis soumises à une charge CPU artificielle avec l'outil `stress-ng`. Le service CLBS a détecté la surcharge et a déclenché une migration live vers un nœud moins chargé.

```
stress-ng --cpu 8
```

Figure IV.17 – Génération d'une charge CPU de test

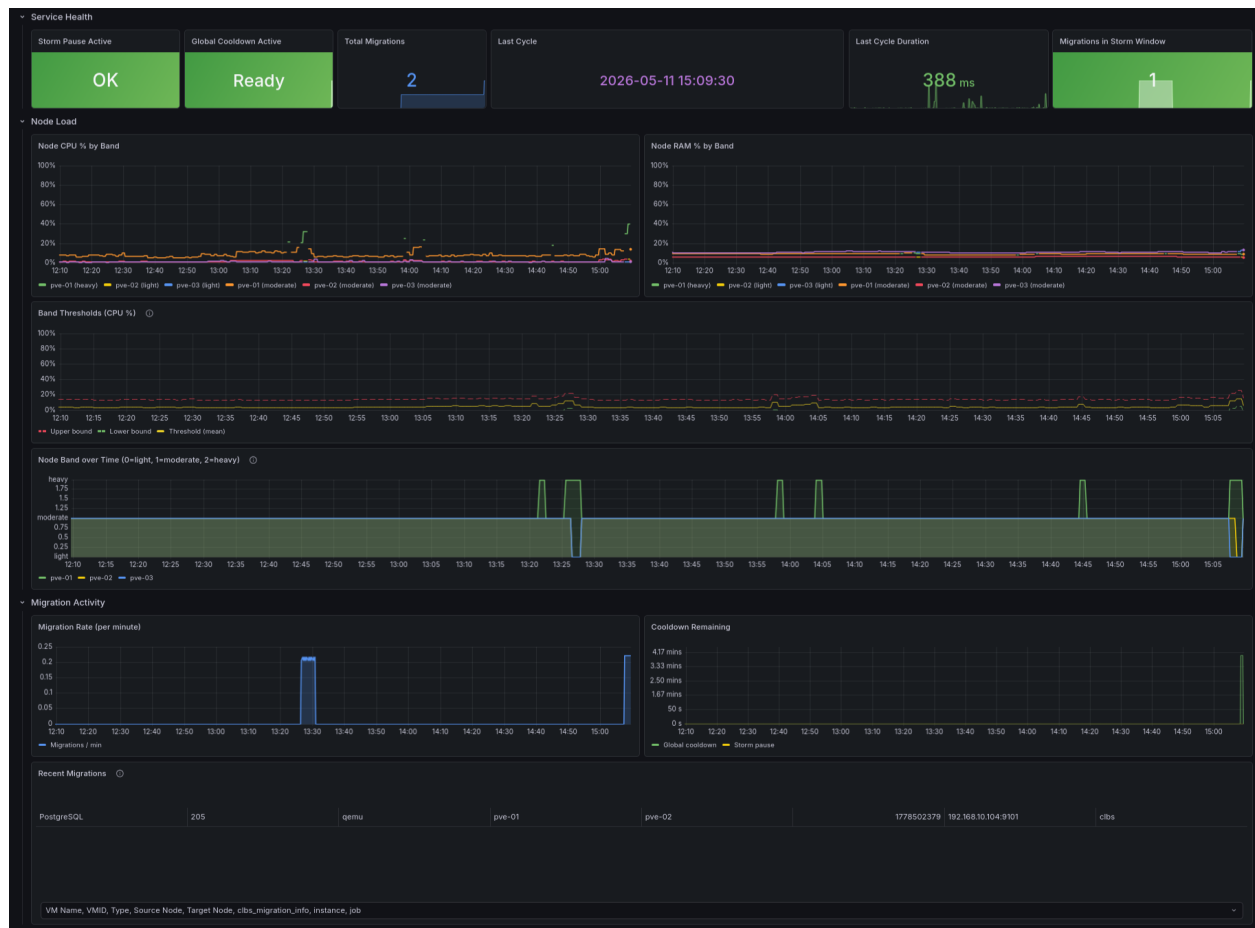


FIGURE IV.18 – Dashboard ou logs montrant une migration déclenchée par CLBS

IV.2.5 Déploiement de l'application Sona-Web

IV.2.5.1 Création des machines virtuelles

L'application Sona-Web a été déployée sous la forme de plusieurs VMs créées à partir d'un template Debian stocké dans le pool Ceph. Cette architecture comprend une VM HA-Proxy en frontal, deux VMs Flask pour la couche applicative, et une VM PostgreSQL pour la base de données.

Les VMs ont été déployées sur les réseaux internes appropriés, avec attribution d'adresse IP par DHCP selon la zone de sécurité visée. Cette organisation permet de séparer la partie exposée, la partie applicative et la base de données.

IV.2.5.2 Validation de l'application

La validation a porté sur le fonctionnement du load balancing ainsi que sur la disponibilité de l'application. Des requêtes répétées à travers HAProxy ont montré une alternance correcte entre les serveurs applicatifs Flask, confirmant le bon fonctionnement du mécanisme de répartition de charge.

```
for i in $(seq 1 10); do
  curl -s http://172.16.80.x | grep "Server"
done
```

Figure IV.19 – Test de répartition HAProxy

IV.2.6 Automatisation avec Ansible

IV.2.6.1 Environnement et structure du projet

Afin de simplifier le provisionnement des machines virtuelles et de rendre l'infrastructure plus reproductible, un environnement Ansible a été mis en place. Les connexions se font via SSH depuis les postes des développeurs vers le réseau de management, sans installation d'agent sur les nœuds cibles.

```
ansible-infra/
├── ansible.cfg
├── secrets.yml
├── inventory/
│   ├── hosts.yml
│   └── proxmox_nodes.yml
├── playbooks/
│   ├── provision_vm.yml
│   └── provision_vm_batch.yml
```

Figure IV.20 – Structure du projet Ansible

```
[defaults]
inventory          = inventory/hosts.yml
remote_user        = ansible
private_key_file    = ~/.ssh/ansible_ed25519
host_key_checking   = False
interpreter_python   = /usr/bin/python3

[privilege_escalation]
become              = True
become_user          = root
```

Figure IV.21 – Fichier ansible.cfg

IV.2.6.2 Inventaire

L'inventaire regroupe les nœuds Proxmox, les VMs permanentes, ainsi qu'un groupe dynamique destiné aux nouvelles VMs provisionnées par les playbooks.

```
all:
  children:
    proxmox_nodes:
      hosts:
        pve-01: {ansible_host: 192.168.10.100}
        pve-02: {ansible_host: 192.168.10.213}
        pve-03: {ansible_host: 192.168.10.84}
    vms:
      hosts:
        monitoring:
```

```
ansible_host: 192.168.10.99
provisioned_vms:
  hosts: {}
```

Figure IV.22 – Extrait de l'inventaire hosts.yml

IV.2.6.3 Playbook de provisionnement

Le playbook principal automatise la création d'une VM à partir d'un template Debian déjà préparé. Il configure les ressources matérielles, le réseau, le cloud-init, le démarrage de la VM, puis sa validation après boot.

```
ansible-playbook playbooks/provision_vm.yml \
-e "vm_id=110 vm_name=web-01"

ansible-playbook playbooks/provision_vm.yml \
-e "vm_id=115 vm_name=db-01 vm_cores=4 vm_memory=8192"
```

Figure IV.23 – Exemple d'utilisation du playbook provision_vm.yml

L'adresse IP de la VM peut être générée automatiquement à partir du VMID pour simplifier le déploiement.

```
vm_ip: "172.16.60.{{ vm_id | int - 90 }}"
```

Figure IV.24 – Formule d'attribution automatique de l'adresse IP

La configuration cloud-init repose sur trois commandes successives : attachement du disque cloud-init, injection de la configuration réseau, puis régénération de l'ISO cloud-init.

```
qm set {{ vm_id }} --ide2 {{ ceph_pool }}:cloudinit
qm set {{ vm_id }} --ipconfig0 ip={{ vm_cidr }},gw={{ vm_gw }} --
  nameserver {{ vm_dns }}
qm cloudinit update {{ vm_id }}
```

Figure IV.25 – Configuration cloud-init d'une VM Proxmox

IV.2.6.4 Provisionnement en lot et pipeline PXE/TFTP

Un second playbook permet le provisionnement en lot de plusieurs VMs à partir d'une liste définie à l'avance. En complément, un pipeline PXE/TFTP a été prévu pour automatiser à terme l'installation de nouveaux nœuds physiques Proxmox sans intervention manuelle, ce qui améliore la capacité de montée en charge de la plateforme.

IV.3 Partie II — Couche Stockage

IV.3.1 Déploiement du cluster Ceph

IV.3.1.1 Initialisation de Ceph

La couche stockage repose sur un cluster Ceph intégré directement à Proxmox. L'initialisation a été effectuée depuis l'interface web ou via la ligne de commande, en définissant un

réseau cluster dédié et un réseau public séparé.

```
pveceph init --network 10.10.30.0/24
```

Figure IV.26 – Initialisation du cluster Ceph

Le réseau 10.10.30.0/24 a été réservé aux échanges internes Ceph, tandis que le réseau 10.10.40.0/24 a été utilisé comme réseau public.

IV.3.1.2 Déploiement des MON et MGR

Un moniteur Ceph a été créé sur chaque nœud du cluster afin d’assurer le quorum et la haute disponibilité des métadonnées du cluster.

```
pveceph mon create
```

Figure IV.27 – Création d’un monitor Ceph

Des managers Ceph ont aussi été déployés pour fournir les fonctions d’administration avancée et l’export des métriques vers Prometheus.

```
ceph mgr module enable prometheus
```

Figure IV.28 – Activation du module Prometheus pour Ceph

IV.3.1.3 Déploiement des OSDs

Chaque nœud a reçu un disque dédié transformé en OSD afin de participer au stockage distribué. Le déploiement a été réalisé sur /dev/sdb pour chacun des trois serveurs.

```
pveceph osd create /dev/sdb
```

Figure IV.29 – Création d’un OSD Ceph

Au total, le cluster dispose de trois OSDs, distribués sur les trois nœuds physiques. La hiérarchie CRUSH a ensuite été vérifiée pour confirmer la bonne prise en compte de tous les nœuds de stockage.

```
ceph osd tree
```

Figure IV.30 – Affichage de l’arbre CRUSH

IV.3.1.4 Création du pool de stockage

Un pool RBD a été créé avec une réplication de taille 3 et une taille minimale de 2, ce qui garantit la disponibilité des données même en cas de perte d’un nœud.

```
pveceph pool create ceph-pool --size 3 --min-size 2
```

Figure IV.31 – Création du pool Ceph

Ce pool a ensuite été intégré comme stockage partagé dans Proxmox pour héberger les disques des VMs et des conteneurs.

```
ceph -s
ceph df
```

Figure IV.32 – État global du cluster Ceph

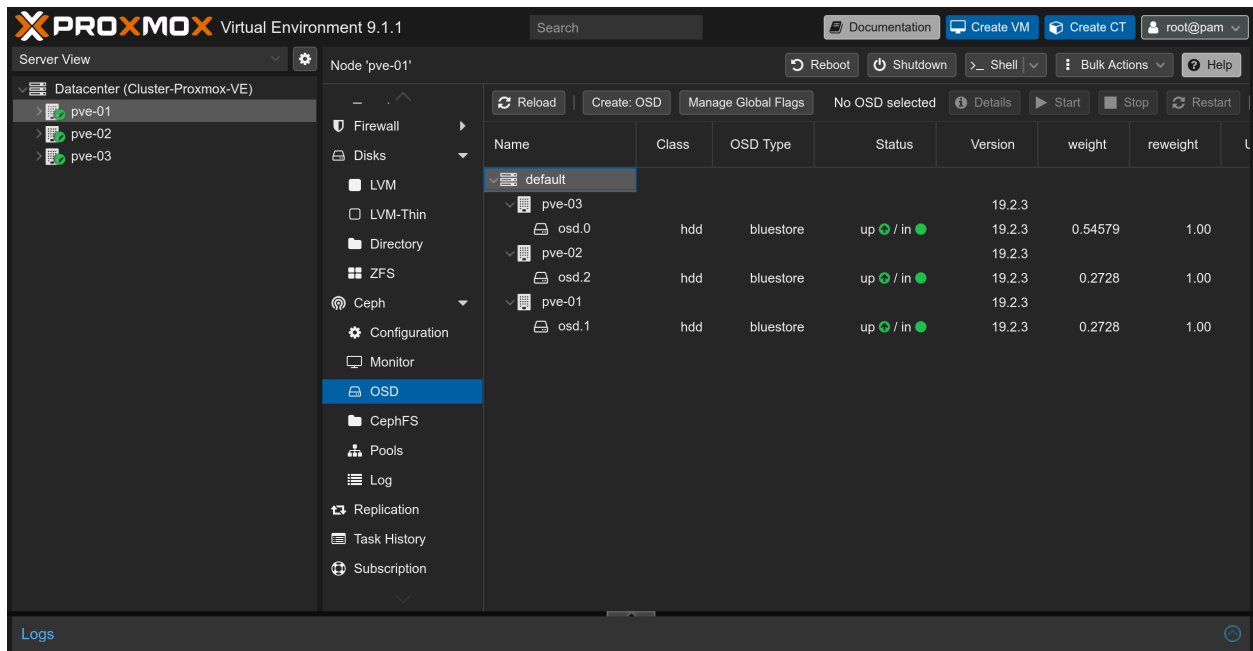


FIGURE IV.33 – État du cluster Ceph montrant les MON et OSD

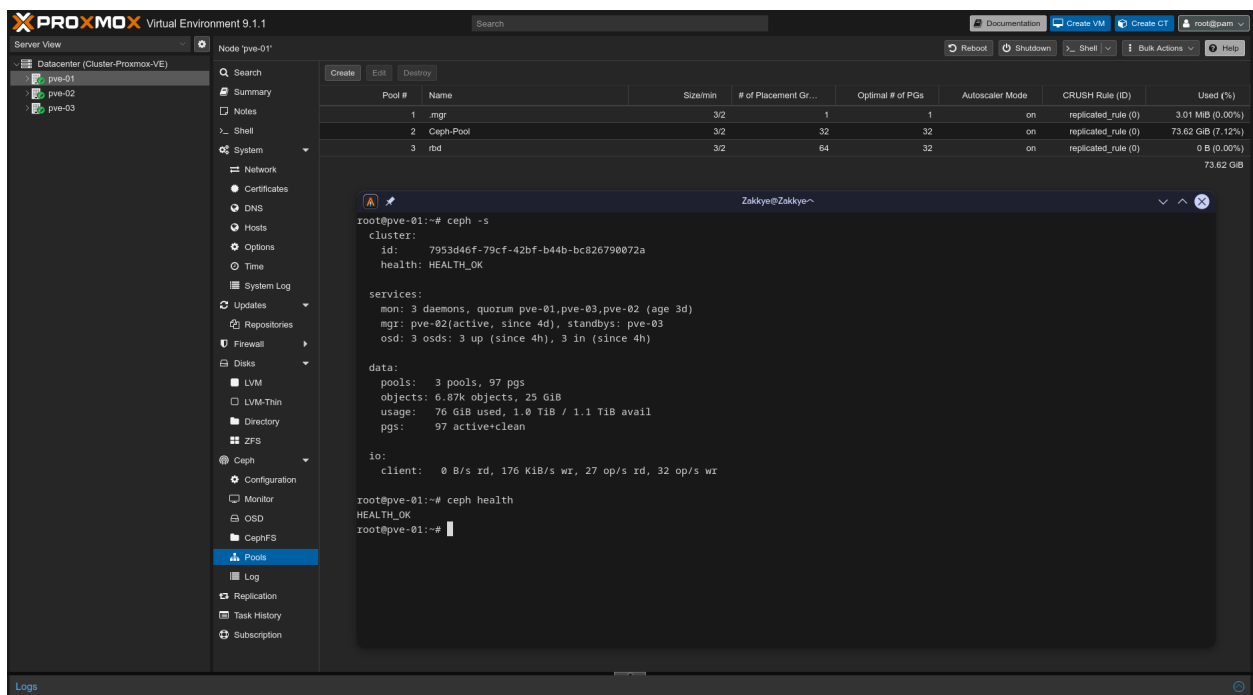


FIGURE IV.34 – État du cluster Ceph montrant le pool RBD

IV.3.2 Validation de la tolérance aux pannes

La tolérance aux pannes de la couche stockage a été vérifiée par un scénario de panne simulée. Lors de l’extinction d’un nœud, le cluster Ceph est passé temporairement à l’état `HEALTH_WARN`, mais les données sont restées accessibles grâce au facteur de réplication.

Après le retour du nœud défaillant, Ceph a automatiquement lancé la resynchronisation des données et l’état du cluster est revenu à `HEALTH_OK`. Ce test valide le bon comportement de la couche stockage distribuée.

IV.4 Partie III — Couche Réseau

IV.4.1 Configuration Open vSwitch et tunnels VxLAN

IV.4.1.1 Installation d’Open vSwitch

Open vSwitch a été installé sur chaque nœud Proxmox afin de construire une couche réseau virtuelle flexible, adaptée au transport de plusieurs flux logiques sur le même réseau physique.

```
apt install openvswitch-switch
```

Figure IV.35 – Installation d’Open vSwitch

Un bridge principal a été créé sur chaque nœud, puis l’interface physique principale y a été attachée pour servir de point de transit à tous les tunnels VxLAN.

```
ovs-vsctl add-br vbr1
ovs-vsctl add-port vbr1 eno1
```

Figure IV.36 – Création du bridge OVS principal

IV.4.1.2 Configuration complète des interfaces sur PVE-01

Le fichier `/etc/network/interfaces` du nœud PVE-01 centralise l’ensemble de la configuration réseau, y compris le bridge principal, les interfaces VxLAN point-à-point vers les autres nœuds, les interfaces internes OVS, et les bridges Linux utilisés pour connecter les machines virtuelles.

```
auto lo
iface lo inet loopback

iface nic0 inet manual

auto vbr0
iface vbr0 inet static
    address 192.168.10.100/24
    gateway 192.168.10.36
    ovs_type OVSBridge
    ovs_ports nic0
```

```
auto nic0
iface nic0 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0

iface nic1 inet manual
iface nic2 inet manual
iface nic3 inet manual

# ===== COROSYNC VXLAN =====
auto vxlan20-pve02
iface vxlan20-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=20

auto vxlan20-pve03
iface vxlan20-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=20

auto vxlan20-int
iface vxlan20-int inet static
    address 10.10.20.1/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== CEPH VXLAN =====
auto vxlan30-pve02
iface vxlan30-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=30

auto vxlan30-pve03
iface vxlan30-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=30

auto vxlan30-int
iface vxlan30-int inet static
    address 10.10.30.1/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 40 (Ceph Public) =====
```

```
auto vxlan40-pve02
iface vxlan40-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=40

auto vxlan40-pve03
iface vxlan40-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=40

auto vxlan40-int
iface vxlan40-int inet static
    address 10.10.40.1/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 50 (Migration) =====
auto vxlan50-pve02
iface vxlan50-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=50

auto vxlan50-pve03
iface vxlan50-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=50

auto vxlan50-int
iface vxlan50-int inet static
    address 10.10.50.1/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 90 (Backup) =====
auto vxlan90-pve02
iface vxlan90-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=90

auto vxlan90-pve03
iface vxlan90-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
```

```
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=90

auto vxlan90-int
iface vxlan90-int inet static
    address 10.10.90.1/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 60 =====
auto vxlan60-pve02
iface vxlan60-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=60

auto vxlan60-pve03
iface vxlan60-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=60

auto vxlan60-int
iface vxlan60-int inet static
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 70 (Dev) =====
auto vxlan70-pve02
iface vxlan70-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=70

auto vxlan70-pve03
iface vxlan70-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=70

auto vxlan70-int
iface vxlan70-int inet static
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 80 (DMZ) =====
auto vxlan80-pve02
iface vxlan80-pve02 inet manual
    ovs_type OVSPort
```

```
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=80

auto vxlan80-pve03
iface vxlan80-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=80

auto vxlan80-int
iface vxlan80-int inet static
    ovs_type OVSIntPort
    ovs_bridge vmbr0

# ===== VM Bridges =====
auto vmbr60
iface vmbr60 inet manual
    bridge_ports vxlan60-int
    bridge_stp off
    bridge_fd 0

auto vmbr70
iface vmbr70 inet manual
    bridge_ports vxlan70-int
    bridge_stp off
    bridge_fd 0

auto vmbr80
iface vmbr80 inet manual
    bridge_ports vxlan80-int
    bridge_stp off
    bridge_fd 0

source /etc/network/interfaces.d/*
```

Figure IV.37 – Configuration réseau complète de PVE-01

Cette architecture permet de transporter plusieurs réseaux logiques isolés sur le même réseau physique sans dépendre d'un switch physique configuré en VLAN.

TABLE IV.2 – Résumé des VxLAN utilisés dans l'infrastructure

VNI	Usage	Réseau
20	Corosync / Heartbeat	10.10.20.0/24
30	Ceph Cluster	10.10.30.0/24
40	Ceph Public	10.10.40.0/24
50	Live Migration	10.10.50.0/24
60	Réseau Dev	172.16.60.0/24
70	Réseau Prod	172.16.70.0/24
80	Réseau DMZ	172.16.80.0/24
90	Backup	10.10.90.0/24

IV.4.2 Déploiement de la VM OPNsense

IV.4.2.1 Création et interfaces réseau

OPNsense a été déployé sous forme de machine virtuelle sur le cluster Proxmox. Son disque système est stocké sur le pool Ceph, ce qui permet de bénéficier du stockage distribué et de la haute disponibilité.

La VM dispose de 4 Go de RAM, de 2 vCPUs et d'un disque de 20 Go. Quatre interfaces réseau lui ont été attribuées afin de jouer le rôle de routeur et pare-feu pour les différents segments de l'infrastructure.

TABLE IV.3 – Interfaces de la VM OPNsense

Interface VM	Bridge Proxmox	Rôle	Adresse IP
net0 (em0)	vmbr0	WAN	192.168.10.65/24
net1 (em1)	vmbr60	LAN Dev	172.16.60.1/24
net2 (em2)	vmbr70	LAN Prod	172.16.70.1/24
net3 (em3)	vmbr80	LAN DMZ	172.16.80.1/24

```

Enter a host name or IP address: 8.8.8.8

PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: icmp_seq=0 ttl=111 time=79.301 ms
^C
*** OPNsense.internal: OPNsense 26.1.2_5 (amd64) ***

LAN (em1)      -> v4: 172.16.60.1/24
OPT1 (em2)     -> v4: 172.16.70.1/24
OPT2 (em3)     -> v4: 172.16.80.1/24
WAN (em0)      -> v4: 192.168.10.65/24

HTTPS: SHA256 DD D5 15 E5 28 38 7B A1 9B E2 1D 5B D4 9C 5D F5
          0B 23 37 EC F7 A8 97 9D F6 CA CE 29 E9 BF 31 D5

0) Logout                7) Ping host
1) Assign interfaces      8) Shell
2) Set interface IP address
3) Reset the root password
4) Reset to factory defaults
5) Power off system
6) Reboot system          9) pfTop
                          10) Firewall log
                          11) Reload all services
                          12) Update from console
                          13) Restore a backup

Enter an option:

```

FIGURE IV.38 – Configuration des interfaces de la VM OPNsense

IV.4.2.2 NAT sortant

Le NAT sortant automatique a été activé dans OPNsense afin de permettre aux réseaux internes de sortir vers Internet via l'interface WAN. Cette configuration a simplifié le routage pour les VMs hébergées sur les réseaux Dev, Prod et DMZ.

IV.4.3 Configuration du service DHCP

Le service DHCP a été activé via `dnsmasq` sur les interfaces LAN internes. Ce choix a été retenu car il est simple à configurer et convient bien à un environnement de laboratoire.

TABLE IV.4 – Plages DHCP configurées sur OPNsense

Interface	Plage début	Plage fin	Gateway
LAN (Dev)	172.16.60.10	172.16.60.250	172.16.60.1
OPT1 (Prod)	172.16.70.10	172.16.70.250	172.16.70.1
OPT2 (DMZ)	172.16.80.10	172.16.80.250	172.16.80.1

Le bon fonctionnement de DHCP a été vérifié par une capture réseau sur l'interface LAN d'OPNsense pendant le démarrage d'une VM de test.

```
tcpdump -i em1 port 67 or port 68
```

Figure IV.39 – Capture du trafic DHCP avec tcpdump

Cette vérification a montré le cycle normal des échanges DHCP, depuis le *Discover* jusqu'au *Offer*, puis l'attribution effective de l'adresse IP à la machine virtuelle.

IV.4.4 Validation de la haute disponibilité réseau et applicative

Un scénario de panne a été réalisé afin de tester le comportement global de l'infrastructure en cas de perte d'un nœud physique. L'arrêt brutal de PVE-01 a permis d'observer la détection de panne par Corosync, le redémarrage des services protégés sur un autre nœud, puis la resynchronisation automatique après le retour du serveur.

Cette séquence a montré que le cluster restait opérationnel malgré la défaillance, et que les mécanismes de haute disponibilité répondaient correctement à ce type d'événement.

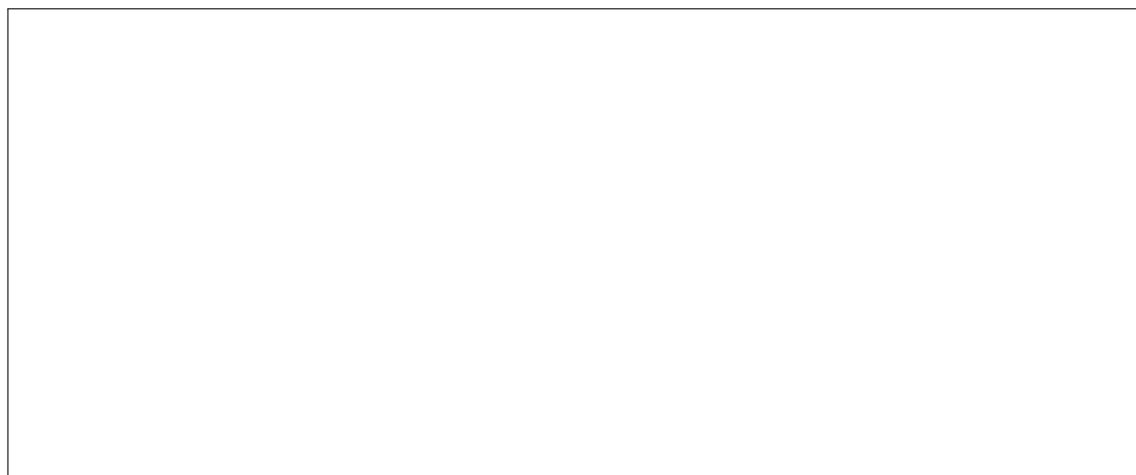


FIGURE IV.40 – Chronologie des événements lors d'une panne d'un nœud physique

IV.5 Conclusion

Ce chapitre a présenté l'implémentation réelle de l'infrastructure hyperconvergée open source mise en place dans le cadre du projet. La couche calcul a permis de déployer Proxmox, le cluster, le monitoring, l'automatisation et les services applicatifs. La couche stockage a été assurée par Ceph avec réplication et tolérance aux pannes. Enfin, la couche réseau a été construite avec Open vSwitch, les tunnels VxLAN, OPNsense et les services d'adressage associés.

Les différents tests réalisés ont confirmé la cohérence de l'architecture proposée et sa capacité à répondre aux objectifs du projet dans un environnement de laboratoire structuré. Le chapitre suivant présentera l'évaluation des performances obtenues ainsi qu'une analyse des résultats expérimentaux.